```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py
然后在浏览器打开 http://127.0.0.1:5000
"""
from app import create_app
from app.models import db
import sys
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")    # 控制台中文正常显示
app = create_app()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()    # 首次运行时自动建表
        from app.utils.schema import ensure_schema
        added = ensure_schema(db)    # 旧库补齐新增字段
        if added:
            print("已补充字段：%s" % "、".join(added))
    print("=" * 58)
    print("  历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ1.0")
    print("=" * 58)
    print("  访问地址：http://127.0.0.1:5000")
    print("  演示账号：admin    密码：admin123")
    print("  停止运行：按 Ctrl+C")
    print("=" * 58)
    app.run(host="127.0.0.1", port=5000, debug=True)
# ==== app/__init__.py ====
# -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from flask import Flask
from flask_login import LoginManager
from app.config import Config
from app.models import db, User
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    # 注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report
```

```
app.register_blueprint(auth.bp)
app.register_blueprint(project.bp)
app.register_blueprint(unit.bp)
app.register_blueprint(standard.bp)
app.register_blueprint(constraint.bp)
app.register_blueprint(ahp.bp)
app.register_blueprint(indicator.bp)
app.register_blueprint(evaluate.bp)
app.register_blueprint(report.bp)
# 全局登录保护：除登录页与静态资源外，均需登录
from flask import request, redirect, url_for
from flask_login import current_user, login_user
PUBLIC_ENDPOINTS = {"auth.login", "static"}
@app.before_request
def require_login():
    # 截图模式（仅用于生成说明书配图）：自动以管理员身份登录
    if app.config.get("SCREENSHOT_MODE"):
        if (request.endpoint not in ("auth.login",
"static")
                and not current_user.is_authenticated):
            admin = User.query.filter_by(role="admin").f
irst()
            if admin:
                login_user(admin)
        return None
    if request.endpoint in PUBLIC_ENDPOINTS or
request.endpoint is None:
        return None
    if not current_user.is_authenticated:
        return redirect(url_for("auth.login"))
    return None
# 侧边栏导航项：集中定义，模板只负责渲染；active 用于高亮当前页
@app.context_processor
def inject_nav():
    path = request.path or "/"
    items = [
        ("首页", "/", path == "/"),
        ("项目与档案", "/project/", path.startswith("/project
")),
        ("限制性判定", "/constraint/",
path.startswith("/constraint")),
        ("标准知识库", "/standard/", path.startswith("/standa
rd")),
        ("AHP 权重", "/ahp/", path.startswith("/ahp")),
        ("指标数据", "/indicator/", path.startswith("/indica
tor")),
        ("适宜性评价", "/evaluate/", path.startswith("/evalua
te")),
        ("报告输出", "/report/", path.startswith("/report"))
,
```

```
            ("用户与日志", "/users", path.startswith(("/users",
    "/logs"))),
        ]
        return {"nav_items": items}
    # 操作审计：统一在响应阶段记录成功的写操作（M1 操作日志）
    from app.services import audit
    @app.after_request
    def audit_writes(response):
        if request.method in ("POST", "PUT", "PATCH",
    "DELETE"):
            audit.record(request.endpoint,
    response.status_code)
        return response
    # 错误页：给出可读提示，而不是默认的英文错误页
    from flask import render_template
    @app.errorhandler(403)
    def forbidden(e):
        return render_template("error.html", code=403,
                               message="没有权限执行该操作"), 403
    @app.errorhandler(404)
    def not_found(e):
        return render_template("error.html", code=404,
                               message="请求的页面不存在"), 404
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
from datetime import timedelta
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY",
    "mine-restoration-dev-key")
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
    # 「记住我」：登录时勾选则下发持久 cookie，关闭浏览器后仍可自动登录
    REMEMBER_COOKIE_DURATION = timedelta(days=30)
    REMEMBER_COOKIE_HTTPONLY = True
    REMEMBER_COOKIE_SAMESITE = "Lax"
    # 影像与数据文件上传（M2 遥感影像挂接、M7 批量导入）
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "static",
    "uploads")
    MAX_CONTENT_LENGTH = 32 * 1024 * 1024          # 单次上传上限
    32 MB
    ALLOWED_IMAGE_EXT = {"png", "jpg", "jpeg", "gif",
    "webp", "bmp", "tif", "tiff"}
```

```
    ALLOWED_DATA_EXT = {"csv", "xlsx", "xls"}
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from datetime import datetime
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（M1 用户与权限管理）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 角色：管理员 admin / 评价员 evaluator / 查看者 viewer
    role = db.Column(db.String(20), nullable=False,
    default="viewer")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    def to_dict(self):
        return {"id": self.id, "username": self.username,
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志（M1）。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, index=True)
class Project(db.Model):
    """评价项目（M2 项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
```

```
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    patches = db.relationship("MinePatch",
    backref="project", lazy="dynamic",
                              cascade="all, delete-orphan")
class MinePatch(db.Model):
    """矿山图斑（M2）。"""
    __tablename__ = "mine_patches"
    id = db.Column(db.Integer, primary_key=True)
    project_id = db.Column(db.Integer,
    db.ForeignKey("projects.id"), nullable=False)
    name = db.Column(db.String(128), nullable=False)
    admin_region = db.Column(db.String(128), nullable=True)
    # 所在行政区
    center_x = db.Column(db.Float, nullable=True)
    # 中心经度 / 横坐标
    center_y = db.Column(db.Float, nullable=True)
    # 中心纬度 / 纵坐标
    area_ha = db.Column(db.Float, nullable=True)
    # 图斑面积（公顷）
    mineral_type = db.Column(db.String(64), nullable=True)
    # 开采矿种
    mining_method = db.Column(db.String(64), nullable=True)
    # 开采方式
    stop_years = db.Column(db.Float, nullable=True)
    # 停产年限
    responsible_entity_missing = db.Column(db.Boolean,
    default=True)  # 责任主体是否缺失
    # 遥感底图 / 无人机正射影像挂接（M2）
    image_path = db.Column(db.String(256), nullable=True)
    # 相对 static 的路径
    image_note = db.Column(db.String(256), nullable=True)
    # 影像说明（如来源、时相）
    image_uploaded_at = db.Column(db.DateTime,
    nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    units = db.relationship("MicroUnit", backref="patch",
    lazy="dynamic",
                            cascade="all, delete-orphan")
class ReferenceEcosystem(db.Model):
    """参照生态系统（M2）：标准化基准值，与图斑绑定。"""
```

```
    __tablename__ = "reference_ecosystems"                          251
    id = db.Column(db.Integer, primary_key=True)                    252
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat        253
    ches.id"), nullable=False)                                      254
    coverage_benchmark = db.Column(db.Float, default=75.0)          255
    # 植被覆盖度基准 %                                              256
    shannon_benchmark = db.Column(db.Float, default=0.7)            257
    # Shannon-Wiener 基准                                           258
    dominant_species = db.Column(db.String(256),                    259
    nullable=True)  # 建群种                                       260
    soil_benchmark = db.Column(db.String(256),                      261
    nullable=True)     # 土壤基准                                   262
class Standard(db.Model):                                           263
    """地方标准 / 政策文件（M3 政策与标准知识库）。"""                 264
    __tablename__ = "standards"                                     265
    id = db.Column(db.Integer, primary_key=True)                    266
    province = db.Column(db.String(32), nullable=False)             267
    # 省份                                                          268
    name = db.Column(db.String(256), nullable=False)                269
    # 名称                                                          270
    doc_number = db.Column(db.String(128), nullable=True)           271
    # 标准编号 / 文号                                               272
    publish_date = db.Column(db.String(32), nullable=True)          273
    implement_date = db.Column(db.String(32), nullable=True)        274
    # 类别：地方标准 / 政策文件                                     275
    category = db.Column(db.String(32), nullable=False,             276
    default="地方标准")                                             277
    summary = db.Column(db.Text, nullable=True)                     278
    # 特点概述                                                      279
    rules = db.relationship("StandardRule",                         280
    backref="standard",                                             281
                                cascade="all, delete-orphan")       282
class StandardRule(db.Model):                                       283
    """结构化规则（M3）：将标准条文拆解为可执行规则。"""             284
    __tablename__ = "standard_rules"                                285
    id = db.Column(db.Integer, primary_key=True)                    286
    standard_id = db.Column(db.Integer,                             287
    db.ForeignKey("standards.id"), nullable=False)                  288
    item = db.Column(db.String(128), nullable=False)                289
    # 判定项                                                        290
    operator = db.Column(db.String(8), nullable=False)              291
    # 比较符：> >= < <= ==                                          292
    threshold = db.Column(db.String(64), nullable=True)             293
    # 阈值                                                          294
    condition_type = db.Column(db.String(32),                       295
    default="必要条件")  # 必要条件 / 充分条件                      296
    region = db.Column(db.String(64), nullable=True)                297
    # 适用分区（宁夏分区差异化）                                    298
    note = db.Column(db.Text, nullable=True)                        299
class ConstraintCheck(db.Model):                                    300
```

```
    """限制性判定记录（M4）。"""                                                301
    __tablename__ = "constraint_checks"                                         302
    id = db.Column(db.Integer, primary_key=True)                                303
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat                    304
    ches.id"), nullable=False)                                                  305
    policy_pass = db.Column(db.Boolean, nullable=True)                          306
    # 政策约束                                                                  307
    location_pass = db.Column(db.Boolean, nullable=True)                        308
    # 区位约束                                                                  309
    pollution_pass = db.Column(db.Boolean, nullable=True)                       310
    # 土壤污染                                                                  311
    geo_pass = db.Column(db.Boolean, nullable=True)                             312
    # 地质灾害                                                                  313
    farmland_pass = db.Column(db.Boolean, nullable=True)                        314
    # 耕地属性                                                                  315
    passed = db.Column(db.Boolean, nullable=True)                               316
    # 总体是否通过                                                              317
    reason = db.Column(db.Text, nullable=True)                                  318
    # 不通过原因                                                                319
    checked_at = db.Column(db.DateTime,                                         320
    default=datetime.utcnow)                                                    321
class MicroUnit(db.Model):                                                      322
    """微地质单元（M5）：岩质边坡 / 碎石边坡 / 平台。"""                        323
    __tablename__ = "micro_units"                                               324
    id = db.Column(db.Integer, primary_key=True)                                325
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat                    326
    ches.id"), nullable=False)                                                  327
    unit_type = db.Column(db.String(32), nullable=False)                        328
    # 单元类型                                                                  329
    area = db.Column(db.Float, nullable=True)                                   330
    # 面积                                                                      331
    ratio = db.Column(db.Float, nullable=True)                                  332
    # 面积占比（自动计算）                                                      333
    description = db.Column(db.Text, nullable=True)                             334
    # 现场描述                                                                  335
class Indicator(db.Model):                                                      336
    """指标定义（M6）：指标树，含角色与启用单元类型。"""                        337
    __tablename__ = "indicators"                                                338
    id = db.Column(db.Integer, primary_key=True)                                339
    code = db.Column(db.String(32), unique=True,                                340
    nullable=False)  # 指标编码                                                 341
    name = db.Column(db.String(64), nullable=False)                             342
    # 指标名称                                                                  343
    level = db.Column(db.Integer, nullable=False)                               344
    # 1/2/3 级                                                                  345
    parent_code = db.Column(db.String(32), nullable=True)                       346
    # 父指标编码                                                                347
    # 角色：土壤污染指标同时作为限制性指标与评价指标                            348
    is_constraint = db.Column(db.Boolean, default=False)                        349
    is_evaluative = db.Column(db.Boolean, default=True)                         350
```

```
    unit = db.Column(db.String(32), nullable=True)
    # 单位
    ideal_value = db.Column(db.String(64), nullable=True)
    # 理想值（可配置）
    ideal_type = db.Column(db.String(32), nullable=True)
    # benefit/cost/mid/discrete
    # 启用该指标的微地质单元（逗号分隔）。岩质边坡不启用土壤指标、平台不启用地形指标
    enabled_units = db.Column(db.String(128), nullable=True)
    description = db.Column(db.Text, nullable=True)
    # 取值依据说明
    def units_list(self):
        return [s for s in (self.enabled_units or
    "").split(",") if s]
class WeightScheme(db.Model):
    """权重方案（M6）：按微地质单元类型分别保存。"""
    __tablename__ = "weight_schemes"
    id = db.Column(db.Integer, primary_key=True)
    unit_type = db.Column(db.String(32), nullable=False)
    # 所属单元类型
    name = db.Column(db.String(64), nullable=True)
    # 方案名称
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 1=一级指标 2=二级指标
    is_default = db.Column(db.Boolean, default=False)
    # 是否默认（论文表 3）
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class WeightItem(db.Model):
    """权重明细（M6）：方案内的各项权重。"""
    __tablename__ = "weight_items"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    indicator_code = db.Column(db.String(32),
    nullable=False)        # 指标编码
    weight = db.Column(db.Float, nullable=False)
    # 权重值
class ExpertMatrix(db.Model):
    """专家判断矩阵（M6）：多专家打分的原始矩阵。"""
    __tablename__ = "expert_matrices"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=True)
    # 直接标注适用对象，便于无需先建权重方案即可录入专家矩阵
    unit_type = db.Column(db.String(32), nullable=True)
    # 微地质单元类型
    l1_name = db.Column(db.String(64), nullable=True)
    # 所属一级指标（二级矩阵用）
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
```

```
    level = db.Column(db.Integer, nullable=False,                    401
    default=1)          # 矩阵对应指标层级                            402
    matrix_json = db.Column(db.Text, nullable=False)                 403
    # 判断矩阵（JSON 二维数组）                                       404
    cr = db.Column(db.Float, nullable=True)                          405
    # 该专家矩阵的一致性比例                                          406
    created_at = db.Column(db.DateTime,                              407
    default=datetime.utcnow)                                         408
class IndicatorData(db.Model):                                       409
    """指标实测数据（M7）：按微地质单元组织。"""                     410
    __tablename__ = "indicator_data"                                 411
    id = db.Column(db.Integer, primary_key=True)                     412
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni         413
    ts.id"), nullable=False, unique=True)                            414
    soil_ratio = db.Column(db.Float, nullable=True)                  415
    # 土石比                                                         416
    thickness = db.Column(db.Float, nullable=True)                   417
    # 土层厚度 cm                                                    418
    ph = db.Column(db.Float, nullable=True)                          419
    organic_matter = db.Column(db.Float, nullable=True)              420
    # 有机质 g/kg                                                    421
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null 422
    heavy_metals = db.Column(db.Text, nullable=True)                 423
    coverage = db.Column(db.Float, nullable=True)                    424
    # 植被覆盖度 %                                                   425
    shannon = db.Column(db.Float, nullable=True)                     426
    # Shannon-Wiener 多样性指数                                      427
    slope = db.Column(db.Float, nullable=True)                       428
    # 坡度 °                                                         429
    aspect = db.Column(db.String(16), nullable=True)                 430
    # 坡向：阳坡/阴阳坡/阴坡                                         431
    def heavy_metals_dict(self):                                     432
        return json.loads(self.heavy_metals) if                      433
    self.heavy_metals else {}                                        434
class EvaluationResult(db.Model):                                    435
    """评价结果（M8）。"""                                           436
    __tablename__ = "evaluation_results"                             437
    id = db.Column(db.Integer, primary_key=True)                     438
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat         439
    ches.id"), nullable=False)                                       440
    final_score = db.Column(db.Float, nullable=True)                 441
    # 最终评价因子 F                                                 442
    grade = db.Column(db.String(32), nullable=True)                  443
    # 适宜性等级                                                     444
    measures = db.Column(db.Text, nullable=True)                     445
    # 推荐措施                                                       446
    unit_scores = db.Column(db.Text, nullable=True)                  447
    # 各单元得分 JSON                                                448
    created_at = db.Column(db.DateTime,                              449
    default=datetime.utcnow)                                         450
```

```
class ScoreDetail(db.Model):
    """得分明细（M8）：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import DataRequired, Length
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES
class AHPSelectForm(FlaskForm):
    """M6：选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """M1：登录表单（占位，由乙完善）。"""
    username = StringField("用户名", validators=[DataRequired()
```

```
    , Length(max=64)])
    password = PasswordField("密码", validators=[DataRequired(
    )])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
# ==== app/utils/constants.py ====
# -*- coding: utf-8 -*-
"""评价指标常量表。
全部数值取自论文，作为系统计算的唯一基准：
- 论文 4.2.2 节：指标标准化理想极值与取值依据
- 论文 4.2.3 节 / 表 4：适宜性等级划分
- 论文表 3：按微地质单元分别确定的一级、二级指标权重
- 论文 4.2.1 节：平均随机一致性指标 RI
"""
# ------------------------------------------------------------------
    ----------
# 一、微地质单元类型
# ------------------------------------------------------------------
    ----------
# 论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")
# 每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
UNIT_ENABLED_INDICATORS = {
    "岩质边坡": ("植被指标", "地形指标"),          # 不设土壤指标
    "碎石边坡": ("土壤指标", "植被指标", "地形指标"),
    "平台": ("土壤指标", "植被指标"),              # 不设地形指标
}
# ------------------------------------------------------------------
    ----------
# 二、指标标准化理想极值（论文 4.2.2 节）
# ------------------------------------------------------------------
    ----------
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 (cm)
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）
IDEAL_ORGANIC = 30.0            # 有机质理想值
    (g/kg)，全国第二次土壤普查养分等级划分标准二等最低值
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 (%)
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统
# 重金属极值 (mg/kg)，《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值
# 论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",
    "Zn")
HEAVY_METAL_LIMIT = {
    "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,
    "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,
}
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）
```

```
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0
# 坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号
SLOPE_BREAK = 25.0
# 坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")
# pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
PH_TOL_LOW = 4.0
PH_TOL_HIGH = 10.0
# ----------------------------------------------------------------------
    ------------------
# 三、适宜性等级划分（论文表 4）
# ----------------------------------------------------------------------
    ------------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按论文表 4 判定适宜性等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ----------------------------------------------------------------------
    ------------------
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2,3,4,5，其余为常用扩展值）
# ----------------------------------------------------------------------
    ------------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
    1.24,
            7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）
CR_THRESHOLD = 0.1
# ----------------------------------------------------------------------
    ------------------
```

```
#  五、默认权重方案（论文表  3）
#  ----------------------------------------------------------------------
    ------------------
#  论文明确："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，
#  分别计算评价指标权重"。权重由  １０  位专家打分后经  ＡＨＰ  方法确定。
#  下表为论文表  3  所载的归一化权重，作为系统默认方案。
ＤＥＦＡＵＬＴ＿ＷＥＩＧＨＴＳ  =  {
    "岩质边坡":  {
        "ｌ１":  {"植被指标":  ０．４５，  "地形指标":  ０．５５},
        "ｌ２":  {
            "植被指标":  {"植被覆盖度":  ０．６７，  "生物多样性":  ０．３３},
            "地形指标":  {"坡度":  ０．６４，  "坡向":  ０．３６},
        },
    },
    "碎石边坡":  {
        "ｌ１":  {"土壤指标":  ０．４４，  "植被指标":  ０．３１，  "地形指标":  ０．２５},
        "ｌ２":  {
            "土壤指标":  {"土石比":  ０．２５，  "土层厚度":  ０．２５，  "ｐＨ":  ０．１７，
                          "有机质":  ０．１７，  "污染指标":  ０．１７},
            "植被指标":  {"植被覆盖度":  ０．６４，  "生物多样性":  ０．３６},
            "地形指标":  {"坡度":  ０．６７，  "坡向":  ０．３３},
        },
    },
    "平台":  {
        "ｌ１":  {"土壤指标":  ０．５４，  "植被指标":  ０．４６},
        "ｌ２":  {
            "土壤指标":  {"土石比":  ０．２５，  "土层厚度":  ０．２５，  "ｐＨ":  ０．１７，
                          "有机质":  ０．１７，  "污染指标":  ０．１７},
            "植被指标":  {"植被覆盖度":  ０．６１，  "生物多样性":  ０．３９},
        },
    },
}
#  ----------------------------------------------------------------------
    ------------------
#  六、限制性指标（论文表  2  +  ４．１  节流程）
#  ----------------------------------------------------------------------
    ------------------
#  论文摘要与  ４．１  节：四项限制性指标；流程段另明确"是否为耕地"为约束条件
ＣＯＮＳＴＲＡＩＮＴ＿ＩＴＥＭＳ  =  （"政策"，  "区位约束"，  "土壤污染"，  "地质灾害"，  "耕地属性"）
#  ====  ａｐｐ/ｓｅｒｖｉｃｅｓ/ａｈｐ＿ｓｅｒｖｉｃｅ．ｐｙ  ====
#  -*-  ｃｏｄｉｎｇ:  ｕｔｆ-８  -*-
"""ＡＨＰ  权重引擎（Ｍ６）。
严格按论文  ４．２．１  节实现：
-  式（２）（３）（４）：特征根法求权重（行元素乘积  →  ｎ  次方根  →  归一化）
-  式（５）（６）（７）：一致性检验（λｍａｘ、ＣＩ、ＣＲ）
-  论文要求  ＣＲ  <  ０．１；ＲＩ  按阶数查表
"""
ｉｍｐｏｒｔ  ｎｕｍｐｙ  ａｓ  ｎｐ
ｆｒｏｍ  ａｐｐ.ｕｔｉｌｓ.ｃｏｎｓｔａｎｔｓ  ｉｍｐｏｒｔ  ＲＩ＿ＴＡＢＬＥ，  ＣＲ＿ＴＨＲＥＳＨＯＬＤ
ｄｅｆ  ｍａｔｒｉｘ＿ｆｒｏｍ＿ｕｐｐｅｒ(ｕｐｐｅｒ):
```

```
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict:{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
```

```
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
    返回
    ----
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)
    及对应取值。
    """
    A = np.asarray(A, dtype=float)
    r = weights_and_consistency(A)
    w = r["weights"]
    n = A.shape[0]
    best = None
    for i in range(n):
        for j in range(i + 1, n):
            expected = w[i] / w[j]
            if expected == 0:
                continue
            deviation = abs(A[i, j] - expected) / expected
            if best is None or deviation >
    best["deviation"]:
                best = {"i": i, "j": j, "value": A[i, j],
                        "expected": expected, "deviation":
    deviation}
    return best
def aggregate_matrices(matrices):
    """多专家判断矩阵聚合：几何平均（保持互反性）。
    参数
    ----
    matrices : list[np.ndarray]，多个同阶判断矩阵。
    """
    stack = np.stack([np.asarray(m, dtype=float) for m in
    matrices])
    return np.prod(stack, axis=0) ** (1.0 / len(stack))
def judge_from_weights(weights):
    """由权重向量构造完全一致判断矩阵并检验。
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。
    """
    w = np.asarray(weights, dtype=float)
    n = w.shape[0]
    A = np.outer(w, 1.0 / w)
    r = weights_and_consistency(A)
    r["matrix"] = A
    return r
# ==== app/services/standardization.py ====
# -*- coding: utf-8 -*-
"""指标标准化（M8 的一部分）。
按论文 4.2.2 节式(8) 实现：
    x' = (x - x_min) / (x_max - x_min)
```

```
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。
所有理想极值取自 app.utils.constants，可配置。
"""
from app.utils.constants import (
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,
    PH_TOL_LOW, PH_TOL_HIGH,
)
def std_benefit(value, ideal):
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, value / ideal))
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染（=1）处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    """
```

```
    total = 0.0                                                          801
    for metal, extreme in HEAVY_METAL_LIMIT.items():                     802
        value = heavy_metals.get(metal)                                  803
        total += std_cost(value, extreme) if value is not                804
    None else 1.0                                                        805
    return total / len(HEAVY_METAL_LIMIT)                                806
def standardize_unit(unit_type, data, weights=None):                     807
    """对单个微地质单元的全部指标做标准化。                                  808
    参数                                                                 809
    ----                                                                 810
    unit_type : str，岩质边坡 / 碎石边坡 / 平台                            811
    data : dict，实测数据，字段见接口契约：                                 812
        soil_ratio, thickness, ph, organic_matter,                       813
    heavy_metals(dict),                                                  814
        coverage, shannon, slope, aspect                                 815
    weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIG     816
    HTS 结构），                                                          817
            未提供时用默认值。                                            818
    返回                                                                 819
    ----                                                                 820
    dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }   821
    标准化值可能为 None，表示该指标不适用或方法待定。                       822
    """                                                                  823
    from app.utils.constants import DEFAULT_WEIGHTS                      824
    if weights is None:                                                  825
        weights = DEFAULT_WEIGHTS[unit_type]                             826
    l1_enabled = weights["l1"].keys()                                    827
    l2w = weights["l2"]                                                  828
    result = {}                                                          829
    # ---- 土壤指标 ----                                                 830
    if "土壤指标" in l1_enabled:                                          831
        result["土壤指标"] = {                                            832
            "土石比": (data.get("soil_ratio"),                           833
                       std_benefit(data.get("soil_ratio"),               834
    IDEAL_SOIL_RATIO),                                                   835
                       l2w["土壤指标"]["土石比"]),                        836
            "土层厚度": (data.get("thickness"),                          837
                         std_benefit(data.get("thickness"),              838
    IDEAL_THICKNESS),                                                    839
                         l2w["土壤指标"]["土层厚度"]),                    840
            "pH": (data.get("ph"),                                       841
                   std_mid_ph(data.get("ph"),                            842
    *IDEAL_PH_RANGE),                                                    843
                   l2w["土壤指标"]["pH"]),                               844
            "有机质": (data.get("organic_matter"),                       845
                       std_benefit(data.get("organic_matter")            846
    , IDEAL_ORGANIC),                                                    847
                       l2w["土壤指标"]["有机质"]),                       848
            "污染指标": (data.get("heavy_metals") or {},                 849
                         pollution_score(data.get("heavy_meta            850
```

```
        ls") or {}),                                                    851
                                    l2w["土壤指标"]["污染指标"]),          852
            }                                                           853
        # ---- 植被指标 ----                                           854
        if "植被指标" in l1_enabled:                                    855
            result["植被指标"] = {                                      856
                "植被覆盖度": (data.get("coverage"),                    857
                                    std_benefit(data.get("coverage"),   858
        IDEAL_COVERAGE),                                                859
                                    l2w["植被指标"]["植被覆盖度"]),      860
                "生物多样性": (data.get("shannon"),                     861
                                    std_benefit(data.get("shannon"),    862
        IDEAL_SHANNON),                                                 863
                                    l2w["植被指标"]["生物多样性"]),      864
            }                                                           865
        # ---- 地形指标 ----                                           866
        if "地形指标" in l1_enabled:                                    867
            result["地形指标"] = {                                      868
                "坡度": (data.get("slope"),                             869
                        std_slope(data.get("slope")),                   870
                        l2w["地形指标"]["坡度"]),                       871
                "坡向": (data.get("aspect"),                            872
                        aspect_score(data.get("aspect")),               873
                        l2w["地形指标"]["坡向"]),                       874
            }                                                           875
        return result                                                   876
# ==== app/services/evaluation_service.py ====                        877
# -*- coding: utf-8 -*-                                               878
"""适宜性评价引擎（M8）。                                               879
按论文式(9) 实现：                                                      880
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）          881
- 矿山级：F = Σ（Si × F'i），Si 为各微地质单元面积占比                  882
- 等级判定按论文表 4。                                                  883
"""                                                                     884
from app.utils.constants import DEFAULT_WEIGHTS, grade_of               885
from app.services import standardization as std                         886
def score_unit(unit_type, data, weights=None):                          887
    """计算单个微地质单元的评价因子 F'。                                888
    返回                                                                889
    ----                                                                890
    dict：{ l1_scores: {一级指标: 得分}, unit_score: F', details:       891
    [...] }                                                             892
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，           893
    则按其余项权重归一化后计算，保证单元得分始终有定义。                894
    """                                                                 895
    if weights is None:                                                 896
        weights = DEFAULT_WEIGHTS[unit_type]                            897
    std_result = std.standardize_unit(unit_type, data,                  898
    weights)                                                            899
    l1_weights = weights["l1"]                                          900
```

```
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
raw,
                    "std_value": None, "weight": w2,
"contribution": None,
                })
                continue
            contribution = std_value * w2
            acc += contribution
            wsum += w2
            details.append({
                "l1": l1_name, "l2": l2_name, "raw": raw,
                "std_value": std_value, "weight": w2,
                "contribution": contribution,
            })
        l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
    unit_score = sum(l1_weights[k] * l1_scores[k] for k in
l1_scores)
    return {"l1_scores": l1_scores, "unit_score":
unit_score, "details": details}
def score_mine(units):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    返回
    ----
    dict:{ final_score, grade, grade_info, unit_results }
    """
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"])
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
            "weighted": weighted,
```

```
                "l1_scores": r["l1_scores"],
                "details": r["details"],
            })
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成（M9）：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
from datetime import datetime
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
    for i, htext in enumerate(headers):
```

```
            _set_run(hdr[i].paragraphs[0].add_run(), htext,
    10.5, bold=True)
    for row in rows:
        cells = t.add_row().cells
        for i, val in enumerate(row):
            _set_run(cells[i].paragraphs[0].add_run(),
    str(val), 10.5)
    return t
def build_report(patch, mine_result):
    """生成评价报告 Word 文档，返回 BytesIO。
    参数
    ----
    patch : MinePatch，图斑对象
    mine_result : evaluation_service.score_mine 的返回 dict
    """
    doc = Document()
    # 页边距
    for section in doc.sections:
        section.top_margin = Cm(2.5)
        section.bottom_margin = Cm(2.5)
        section.left_margin = Cm(2.5)
        section.right_margin = Cm(2.5)
    _heading(doc, "历史遗留废弃矿山自然恢复适宜性评价报告", 0)
    _para(doc, "软件：历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ1.0",
          size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER)
    _para(doc, "生成时间：%s" % datetime.now().strftime("%Y-%m-%d
    %H:%M:%S"),
          size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER,
    space_after=12)
    # 图斑信息
    _heading(doc, "一、图斑基本信息", 1)
    info = [
        ("图斑名称", patch.name),
        ("所在行政区", patch.admin_region or "—"),
        ("图斑面积（公顷）", str(patch.area_ha) if patch.area_ha is
    not None else "—"),
        ("开采矿种", patch.mineral_type or "—"),
        ("开采方式", patch.mining_method or "—"),
    ]
    _table(doc, ["项目", "内容"], info)
    # 评价结论
    _heading(doc, "二、评价结论", 1)
    gi = mine_result["grade_info"]
    concl = [
        ("最终评价因子 F", "%.4f" % mine_result["final_score"]),
        ("适宜性等级", mine_result["grade"]),
        ("判定依据", gi["meaning"]),
        ("推荐措施", gi["measure"]),
    ]
    _table(doc, ["项目", "结果"], concl)
```

```
    # 各单元得分
    _heading(doc, "三、各微地质单元得分", 1)
    rows = []
    for u in mine_result["unit_results"]:
        rows.append((u["unit_type"], "%.2f%%" % (u["ratio"] * 100),
                     "%.4f" % u["unit_score"], "%.4f" % u["weighted"]))
    _table(doc, ["微地质单元", "面积占比", "单元评价因子 F'", "加权贡献"], rows)
    # 完整得分明细
    _heading(doc, "四、得分明细", 1)
    for u in mine_result["unit_results"]:
        _para(doc, "%s（占比 %.2f%%）" % (u["unit_type"], u["ratio"] * 100),
              size=11, bold=True, space_after=4)
        detail_rows = []
        for d in u["details"]:
            raw = "—" if d["raw"] is None else d["raw"]
            std = "—" if d["std_value"] is None else "%.4f" % d["std_value"]
            w = "%.4f" % d["weight"]
            c = "—" if d["contribution"] is None else "%.4f" % d["contribution"]
            detail_rows.append((d["l1"], d["l2"], raw, std, w, c))
        _table(doc, ["一级指标", "二级指标", "实测值", "标准化值", "权重", "贡献"],
               detail_rows)
        # 一级指标得分
        l1_row = []
        for k, v in u["l1_scores"].items():
            l1_row.append("%s=%.4f" % (k, v))
        _para(doc, "一级指标得分：" + "，".join(l1_row), size=10.5, space_after=10)
    buf = io.BytesIO()
    doc.save(buf)
    buf.seek(0)
    return buf
# ==== app/views/auth.py ====
# -*- coding: utf-8 -*-
"""M1 用户与权限管理 —— 登录、注销、用户维护、操作日志。"""
from flask import (Blueprint, render_template, redirect,
    url_for, request,
                   flash, abort)
from flask_login import login_user, logout_user,
    login_required, current_user
from werkzeug.security import check_password_hash,
    generate_password_hash
from app.forms import LoginForm, EmptyForm
```

```
from app.models import db, User, OperationLog                    1101
bp = Blueprint("auth", __name__)                                 1102
ROLE_LABELS = {"admin": "管理员", "evaluator": "评价员",          1103
    "viewer": "查看者"}                                          1104
def require_admin():                                             1105
    """非管理员直接 403，避免误操作基础数据。"""                  1106
    if current_user.role != "admin":                             1107
        abort(403)                                               1108
@bp.route("/")                                                   1109
def index():                                                     1110
    if current_user.is_authenticated:                            1111
        return render_template("auth/home.html")                 1112
    return redirect(url_for("auth.login"))                       1113
@bp.route("/login", methods=["GET", "POST"])                     1114
def login():                                                     1115
    if current_user.is_authenticated:                            1116
        return redirect(url_for("auth.index"))                   1117
    form = LoginForm()                                           1118
    if form.validate_on_submit():                                1119
        user = User.query.filter_by(username=form.username.d1120
    ata).first()                                                 1121
        if user and check_password_hash(user.password_hash,      1122
    form.password.data):                                         1123
            if not user.active:                                  1124
                flash("该账号已停用，请联系管理员", "error")      1125
                return render_template("auth/login.html",        1126
    form=form)                                                   1127
            login_user(user, remember=form.remember.data)        1128
            flash("登录成功", "ok")                              1129
            return redirect(url_for("auth.index"))               1130
        flash("用户名或密码错误", "error")                       1131
    return render_template("auth/login.html", form=form)         1132
@bp.route("/logout")                                             1133
@login_required                                                  1134
def logout():                                                    1135
    logout_user()                                                1136
    flash("已退出登录", "ok")                                    1137
    return redirect(url_for("auth.login"))                       1138
# ---------------------------- 用户维护 ----------------------------1139
    ----                                                         1140
@bp.route("/users")                                              1141
@login_required                                                  1142
def users():                                                     1143
    all_users = User.query.order_by(User.id).all()               1144
    return render_template("auth/users.html",                    1145
    users=all_users,                                             1146
                                  role_labels=ROLE_LABELS,       1147
    form=EmptyForm())                                            1148
@bp.route("/users/add", methods=["GET", "POST"])                 1149
@login_required                                                  1150
```

```
def add_user():
    require_admin()
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        role = request.form.get("role", "viewer")
        if not username or not password:
            flash("用户名和密码不能为空", "error")
            return redirect(url_for("auth.add_user"))
        if User.query.filter_by(username=username).first():
            flash("用户名已存在", "error")
            return redirect(url_for("auth.add_user"))
        db.session.add(User(username=username, role=role,
                            password_hash=generate_password_
    hash(password)))
        db.session.commit()
        flash("用户已添加", "ok")
        return redirect(url_for("auth.users"))
    return render_template("auth/add_user.html")
@bp.route("/users/<int:user_id>/edit", methods=["GET",
    "POST"])
@login_required
def edit_user(user_id):
    require_admin()
    user = db.get_or_404(User, user_id)
    if request.method == "POST":
        role = request.form.get("role", user.role)
        active = request.form.get("active") == "1"
        # 保护：不能把最后一个管理员降级或停用，否则无人能管理系统
        if user.role == "admin" and (role != "admin" or not
    active):
            others = User.query.filter(User.role == "admin",
                                       User.id != user.id,

    User.active.is_(True)).count()
            if others == 0:
                flash("系统至少需保留一名启用状态的管理员", "error")
                return redirect(url_for("auth.edit_user",
    user_id=user.id))
        user.role = role
        user.active = active
        new_password = request.form.get("password",
    "").strip()
        if new_password:
            user.password_hash = generate_password_hash(new_
    password)
        db.session.commit()
        flash("用户信息已保存", "ok")
        return redirect(url_for("auth.users"))
    return render_template("auth/edit_user.html", user=user,
```

```
                                role_labels=ROLE_LABELS,
    form=EmptyForm())
@bp.route("/users/<int:user_id>/delete", methods=["POST"])
@login_required
def delete_user(user_id):
    require_admin()
    user = db.get_or_404(User, user_id)
    if user.id == current_user.id:
        flash("不能删除当前登录的账号", "error")
        return redirect(url_for("auth.users"))
    if user.role == "admin":
        others = User.query.filter(User.role == "admin",
                                   User.id != user.id,
                                   User.active.is_(True)).co
    unt()
        if others == 0:
            flash("系统至少需保留一名启用状态的管理员", "error")
            return redirect(url_for("auth.users"))
    db.session.delete(user)
    db.session.commit()
    flash("用户「%s」已删除" % user.username, "ok")
    return redirect(url_for("auth.users"))
# ------------------------------ 操作日志 ------------------------------
    ----
@bp.route("/logs")
@login_required
def logs():
    page = request.args.get("page", 1, type=int)
    per_page = 50
    q = OperationLog.query.order_by(OperationLog.id.desc())
    total = q.count()
    records = q.offset((page - 1) * per_page).limit(per_page
    ).all()
    users_map = {u.id: u.username for u in User.query.all()}
    pages = max(1, (total + per_page - 1) // per_page)
    return render_template("auth/logs.html",
    records=records, page=page,
                           pages=pages, total=total,
    users_map=users_map)
# ==== app/views/project.py ====
# -*- coding: utf-8 -*-
"""M2 项目与矿山档案 —— 项目、图斑、参照生态系统、遥感影像挂接。"""
from datetime import datetime
from flask import (Blueprint, render_template, request,
    redirect, url_for,
                   flash, current_app)
from flask_login import login_required, current_user
from app.forms import EmptyForm
from app.models import db, Project, MinePatch,
    ReferenceEcosystem
```

```
bp = Blueprint("project", __name__, url_prefix="/project")
@bp.route("/")
@login_required
def index():
    projects = Project.query.order_by(Project.id.desc()).all()
    return render_template("project/index.html",
    projects=projects,
                           form=EmptyForm())
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit():
        p = Project(
            name=request.form.get("name", "").strip(),
            code=request.form.get("code", "").strip() or None,
            description=request.form.get("description", "").strip() or None,
            created_by=current_user.id,
        )
        if not p.name:
            flash("项目名称不能为空", "error")
        else:
            db.session.add(p)
            db.session.commit()
            flash("项目已创建", "ok")
            return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form, project=None)
@bp.route("/<int:pid>/edit", methods=["GET", "POST"])
@login_required
def edit(pid):
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit():
        p.name = request.form.get("name", "").strip()
        p.code = request.form.get("code", "").strip() or None
        p.description = request.form.get("description", "").strip() or None
        p.status = request.form.get("status", "进行中")
        db.session.commit()
        flash("项目已保存", "ok")
        return redirect(url_for("project.index"))
    return render_template("project/edit.html", form=form, project=p)
```

```
@bp.route("/<int:pid>/delete", methods=["POST"])
@login_required
def delete(pid):
    p = db.get_or_404(Project, pid)
    db.session.delete(p)
    db.session.commit()
    flash("项目已删除", "ok")
    return redirect(url_for("project.index"))
# ---------------------------- 矿山图斑 ----------------------------
----
@bp.route("/<int:pid>/patches")
@login_required
def patches(pid):
    p = db.get_or_404(Project, pid)
    return render_template("project/patches.html",
    project=p,
                           form=EmptyForm())
@bp.route("/patch/<int:patch_id>/delete", methods=["POST"])
@login_required
def patch_delete(patch_id):
    """删除矿山图斑，及其微地质单元与指标数据。"""
    from app.models import MicroUnit, IndicatorData,
    EvaluationResult, \
        ScoreDetail, ConstraintCheck, ReferenceEcosystem
    patch = db.get_or_404(MinePatch, patch_id)
    pid = patch.project_id
    unit_ids = [u.id for u in MicroUnit.query.filter_by(patc
    h_id=patch.id).all()]
    if unit_ids:
        IndicatorData.query.filter(IndicatorData.unit_id.in_
    (unit_ids)).delete(
            synchronize_session=False)
    for r in EvaluationResult.query.filter_by(patch_id=patch
    .id).all():
        ScoreDetail.query.filter_by(result_id=r.id).delete(
            synchronize_session=False)
    EvaluationResult.query.filter_by(patch_id=patch.id).dele
    te(
        synchronize_session=False)
    ConstraintCheck.query.filter_by(patch_id=patch.id).delet
    e(
        synchronize_session=False)
    MicroUnit.query.filter_by(patch_id=patch.id).delete(sync
    hronize_session=False)
    ReferenceEcosystem.query.filter_by(patch_id=patch.id).de
    lete(
        synchronize_session=False)
    db.session.delete(patch)
    db.session.commit()
    flash("图斑「%s」及其数据已删除" % patch.name, "ok")
```

```
    return redirect(url_for("project.patches", pid=pid))
@bp.route("/<int:pid>/patch/add", methods=["GET", "POST"])
@login_required
def patch_add(pid):
    p = db.get_or_404(Project, pid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch = MinePatch(
            project_id=p.id,
            name=request.form.get("name", "").strip(),
            admin_region=fv("admin_region"),
            area_ha=fv("area_ha", float),
            mineral_type=fv("mineral_type"),
            mining_method=fv("mining_method"),
            stop_years=fv("stop_years", float),
            responsible_entity_missing=bool(request.form.get
    ("responsible_entity_missing")),
        )
        if not patch.name:
            flash("图斑名称不能为空", "error")
        else:
            db.session.add(patch)
            db.session.flush()
            # 参照生态系统默认取论文值
            db.session.add(ReferenceEcosystem(
                patch_id=patch.id, coverage_benchmark=75.0,
    shannon_benchmark=0.7))
            db.session.commit()
            flash("图斑已创建", "ok")
            return redirect(url_for("project.patches",
    pid=p.id))
    return render_template("project/patch_edit.html",
    form=form, project=p, patch=None)
@bp.route("/patch/<int:patch_id>/edit", methods=["GET",
    "POST"])
@login_required
def patch_edit(patch_id):
    patch = db.get_or_404(MinePatch, patch_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch.name = request.form.get("name", "").strip()
        patch.admin_region = fv("admin_region")
```

```
        patch.area_ha = fv("area_ha", float)                          1401
        patch.mineral_type = fv("mineral_type")                       1402
        patch.mining_method = fv("mining_method")                     1403
        patch.stop_years = fv("stop_years", float)                    1404
        patch.responsible_entity_missing =                            1405
    bool(request.form.get("responsible_entity_missing"))              1406
        # 参照生态系统                                                 1407
        ref = ReferenceEcosystem.query.filter_by(patch_id=pa          1408
    tch.id).first()                                                   1409
        if ref is None:                                               1410
            ref = ReferenceEcosystem(patch_id=patch.id)               1411
            db.session.add(ref)                                       1412
        ref.coverage_benchmark = fv("coverage_benchmark",             1413
    float) or 75.0                                                    1414
        ref.shannon_benchmark = fv("shannon_benchmark",               1415
    float) or 0.7                                                     1416
        ref.dominant_species = fv("dominant_species")                 1417
        patch.image_note = fv("image_note")                           1418
        db.session.commit()                                           1419
        flash("图斑已保存", "ok")                                      1420
        return redirect(url_for("project.patches",                    1421
    pid=patch.project_id))                                            1422
    return render_template("project/patch_edit.html",                 1423
    form=form,                                                        1424
                           project=patch.project,                     1425
    patch=patch,                                                      1426
                           reference=ReferenceEcosystem.quer          1427
    y.filter_by(                                                      1428
                               patch_id=patch.id).first())            1429
# ------------------------------ 遥感影像挂接                        1430
    ------------------------------                                    1431
@bp.route("/patch/<int:patch_id>/image", methods=["POST"])            1432
@login_required                                                       1433
def upload_image(patch_id):                                           1434
    """上传遥感底图 / 无人机正射影像并挂接到图斑。"""                    1435
    from app.utils import uploads as up                               1436
    patch = db.get_or_404(MinePatch, patch_id)                        1437
    form = EmptyForm()                                                1438
    if not form.validate_on_submit():                                 1439
        flash("表单校验失败，请重试", "error")                         1440
        return redirect(url_for("project.patch_edit",                 1441
    patch_id=patch.id))                                               1442
    rel, err = up.save_upload(request.files.get("image"),             1443
    "patches",                                                        1444
                              current_app.config["ALLOWED_IM          1445
    AGE_EXT"],                                                        1446
                              current_app.config["UPLOAD_FOL          1447
    DER"])                                                            1448
    if err:                                                           1449
        flash(err, "error")                                           1450
```

```
            return redirect(url_for("project.patch_edit",
        patch_id=patch.id))
        # 替换旧影像时清理原文件，避免残留
        if patch.image_path:
            up.remove_upload(patch.image_path,
        current_app.config["UPLOAD_FOLDER"])
        patch.image_path = rel
        patch.image_uploaded_at = datetime.utcnow()
        note = request.form.get("image_note", "").strip()
        if note:
            patch.image_note = note
        db.session.commit()
        flash("遥感影像已挂接", "ok")
        return redirect(url_for("project.patch_edit",
        patch_id=patch.id))
@bp.route("/patch/<int:patch_id>/image/delete",
        methods=["POST"])
@login_required
def delete_image(patch_id):
        from app.utils import uploads as up
        patch = db.get_or_404(MinePatch, patch_id)
        form = EmptyForm()
        if form.validate_on_submit():
            up.remove_upload(patch.image_path,
        current_app.config["UPLOAD_FOLDER"])
            patch.image_path = None
            patch.image_uploaded_at = None
            db.session.commit()
            flash("遥感影像已移除", "ok")
        return redirect(url_for("project.patch_edit",
        patch_id=patch.id))
# ==== app/views/unit.py ====
# -*- coding: utf-8 -*-
"""M5 微地质单元管理 —— 岩质边坡 / 碎石边坡 / 平台的划分与面积占比。"""
from flask import Blueprint, render_template, request,
        redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, MicroUnit,
        IndicatorData
from app.utils.constants import UNIT_TYPES
bp = Blueprint("unit", __name__, url_prefix="/unit")
def recalc_ratio(patch_id):
        """按面积重算该图斑各单元面积占比。"""
        units = MicroUnit.query.filter_by(patch_id=patch_id).all
        ()
        total = sum(u.area or 0.0 for u in units)
        if total > 0:
            for u in units:
                u.ratio = round((u.area or 0.0) / total, 6)
```

```
        8.42,0.72,,,\n"
        "不存在单元,1,1,7,1,,,,,,,,,,\n"
)
r = c.post("/indicator/import",
                data={"patch_id": str(patch_id),
                      "datafile": (io.BytesIO(csv_text.encode("ut
        f-8")), "d.csv")},
                content_type="multipart/form-data",
        follow_redirects=True)
body = r.get_data(as_text=True)
ok("导入结果" in body, "导入结果页展示")
ok("不存在单元" in body, "错误行被报告")
with app.app_context():
        u = MicroUnit.query.filter_by(patch_id=patch_id,
        unit_type="碎石边坡").first()
        ind = IndicatorData.query.filter_by(unit_id=u.id).first(
        )
        ok(abs(ind.soil_ratio - 0.34) < 1e-6 and
        abs(ind.coverage - 87.92) < 1e-6,
            "导入数据已写入数据库")
r = c.post("/indicator/import",
                data={"patch_id": str(patch_id),
                      "datafile": (io.BytesIO(b"x"), "d.pdf")},
                content_type="multipart/form-data",
        follow_redirects=True)
ok("仅支持" in r.get_data(as_text=True), "拒绝不支持的导入格式")
# ------------------------------------------------------------
        ------------------
print()
print(LINE)
print("M9 图表与 PDF 导出")
print(LINE)
svg = charts.hbars([("岩质边坡", 0.54, "0.5401"), ("平台", 0.84,
        "0.8376")])
ok("<svg" in svg and "0.8376" in svg, "条形图生成且带数值标签")
svg2 = charts.donut([("植被指标", 0.6, "0.6000"), ("土壤指标", 0.4,
        "0.4000")])
ok("<svg" in svg2 and "占比" in svg2, "环形图生成且带图例")
r = c.get("/report/%d" % patch_id)
body = r.get_data(as_text=True)
ok(body.count("<svg") >= 2, "得分明细页含图表", "%d 个 SVG" %
        body.count("<svg"))
exe = browser.browser_path()
if exe is None:
        print("  [跳过] 未找到浏览器，PDF 导出无法验证")
else:
        r = c.get("/report/%d/pdf" % patch_id)
        data = r.data
        ok(data[:4] == b"%PDF", "PDF 报告导出成功", "%d 字节" %
        len(data))
```

```
    d = fitz.open(stream=data, filetype="pdf")
    txt = "".join(d[i].get_text() for i in
    range(d.page_count))
    ok("自然恢复适宜性评价报告" in txt, "PDF 含报告标题")
    ok("评价结论" in txt and "得分明细" in txt, "PDF 含结论与明细章节")
    ok(d.page_count >= 2, "PDF 页数合理", "%d 页" % d.page_count)
    d.close()
print()
print(LINE)
print("补齐模块自检全部通过 √")
print(LINE)
#  ==== app/services/__init__.py ====
#  -*- coding: utf-8 -*-
#  ==== app/services/audit.py ====
#  -*- coding: utf-8 -*-
"""操作审计（M1）。
在应用工厂中以 after_request 钩子统一埋点：凡是成功的写操作（POST/PUT/DELETE）
且已登录，即写入 operation_logs 表，无需在每个视图里手工调用。
动作名称由「端点 → 中文动作」映射表给出，未收录的端点回退为方法 + 路径。
"""
from flask import request
from flask_login import current_user
from app.models import db, OperationLog
#  端点 → 可读动作名
ACTION_NAMES = {
    "auth.login": "登录系统",
    "auth.logout": "退出登录",
    "auth.add_user": "新增用户",
    "auth.edit_user": "编辑用户",
    "auth.delete_user": "删除用户",
    "auth.reset_password": "重置用户密码",
    "project.add": "新建项目",
    "project.edit": "编辑项目",
    "project.delete": "删除项目",
    "project.patch_add": "新建矿山图斑",
    "project.patch_edit": "编辑矿山图斑",
    "project.patch_delete": "删除矿山图斑",
    "project.upload_image": "上传遥感影像",
    "unit.add": "添加微地质单元",
    "unit.edit": "编辑微地质单元",
    "unit.delete": "删除微地质单元",
    "standard.add": "添加标准/政策文件",
    "standard.edit": "编辑标准/政策文件",
    "standard.delete": "删除标准/政策文件",
    "standard.rules": "维护标准规则",
    "standard.rule_delete": "删除标准规则",
    "constraint.check": "执行限制性条件判定",
    "ahp.save_expert": "录入专家判断矩阵",
    "ahp.delete_expert": "删除专家判断矩阵",
    "ahp.aggregate": "汇总多专家矩阵",
```

```
    "indicator.edit": "保存指标数据",
    "indicator.import_data": "批量导入指标数据",
    "indicator.update_meta": "维护指标定义",
    "evaluate.run": "执行适宜性评价",
}
# 无需记录的操作（查看类或纯导航）
SKIP_ENDPOINTS = {"static", "auth.index"}
def action_name(endpoint):
    if endpoint in ACTION_NAMES:
        return ACTION_NAMES[endpoint]
    return "写操作 %s %s" % (request.method, request.path)
def extract_target_id():
    """从视图参数中取出被操作对象的主键，取不到则返回 None。"""
    args = request.view_args or {}
    for key in ("pid", "patch_id", "unit_id", "sid", "rid",
    "result_id", "user_id"):
        if key in args:
            try:
                return int(args[key])
            except (TypeError, ValueError):
                return None
    return None
def record(endpoint, status_code):
    """记录一次写操作。失败（4xx/5xx）也记录，便于追溯。"""
    if endpoint in SKIP_ENDPOINTS:
        return
    try:
        detail = "路径 %s" % request.path
        if status_code >= 400:
            detail += "；结果 HTTP %d" % status_code
        db.session.add(OperationLog(
            user_id=current_user.id if
    current_user.is_authenticated else None,
            action=action_name(endpoint),
            target_type=(endpoint or "").split(".")[0] or
    None,
            target_id=extract_target_id(),
            detail=detail,
        ))
        db.session.commit()
    except Exception:
        # 审计失败不应影响业务响应
        db.session.rollback()
# ==== app/services/import_service.py ====
# -*- coding: utf-8 -*-
"""指标数据批量导入（M7）。
支持 CSV 与 Excel（.xlsx/.xls）。一行对应一个微地质单元，
按「单元类型」匹配图斑内已有的微地质单元后写入指标数据。
表头约定（缺失列按不适用处理）：
    单元类型， 土石比， 土层厚度， pH， 有机质，
```

```
    Hg, As, Ni, Cu, Cd, Pb, Cr, Zn,
    植被覆盖度, Shannon, 坡度, 坡向
"""
import csv
import io
import json
from app.utils.constants import HEAVY_METALS, ASPECT_TYPES,
    UNIT_TYPES
# 表头（顺序即导出模板顺序）
HEADERS = (["单元类型", "土石比", "土层厚度", "pH", "有机质"]
           + list(HEAVY_METALS)
           + ["植被覆盖度", "Shannon", "坡度", "坡向"])
FIELD_ALIASES = {
    "单元类型": "unit_type",
    "土石比": "soil_ratio",
    "土层厚度": "thickness",
    "pH": "ph", "PH": "ph", "ph": "ph",
    "有机质": "organic_matter",
    "植被覆盖度": "coverage",
    "Shannon": "shannon", "shannon": "shannon", "生物多样性":
    "shannon",
    "坡度": "slope",
    "坡向": "aspect",
}
for _m in HEAVY_METALS:
    FIELD_ALIASES[_m] = "hm_" + _m
def build_template_csv():
    """生成导入模板（含表头与示例行）。"""
    buf = io.StringIO()
    w = csv.writer(buf)
    w.writerow(HEADERS)
    # 示例：碎石边坡一行，说明每列含义
    row = {h: "" for h in HEADERS}
    row.update({"单元类型": "碎石边坡", "土石比": "0.34", "土层厚度": "10",
                "pH": "9.24", "有机质": "4.89", "Hg": "0.05",
    "As": "5.17",
                "Ni": "12.60", "Cu": "11.70", "Cd": "0.04",
    "Pb": "6.50",
                "植被覆盖度": "87.92", "Shannon": "0.60",
                "坡度": "28.75", "坡向": "阳坡"})
    w.writerow([row[h] for h in HEADERS])
    return buf.getvalue()
def _read_rows(filename, stream):
    """读取上传文件为「表头 -> 值」的字典列表。"""
    ext = filename.rsplit(".", 1)[-1].lower()
    if ext == "csv":
        text = stream.read().decode("utf-8-sig",
    errors="replace")
        reader = csv.DictReader(io.StringIO(text))
        return [dict(r) for r in reader]
```

```
    if ext in ("xlsx", "xls"):
        from openpyxl import load_workbook
        wb = load_workbook(io.BytesIO(stream.read()),
data_only=True)
        ws = wb.active
        rows = list(ws.iter_rows(values_only=True))
        if not rows:
            return []
        header = [str(c).strip() if c is not None else ""
for c in rows[0]]
        out = []
        for r in rows[1:]:
            if all(c is None or str(c).strip() == "" for c
in r):
                continue
            out.append({header[i]: (r[i] if i < len(r) else
None)
                        for i in range(len(header)) if
header[i]})
        return out
    raise ValueError("不支持的文件类型：.%s" % ext)
def _num(value):
    if value is None:
        return None, None
    s = str(value).strip()
    if s == "":
        return None, None
    try:
        return float(s), None
    except ValueError:
        return None, "「%s」不是有效数值" % s
def parse_upload(filename, stream, unit_type_map):
    """解析上传文件。
    参数
    ----
    unit_type_map : dict, {单元类型: MicroUnit 对象}
    返回
    ----
    (parsed, errors)
      parsed : list[dict], 每项 {unit_type, fields{...},
    heavy_metals{}}
      errors : list[str], 逐行问题说明
    """
    rows = _read_rows(filename, stream)
    parsed, errors = [], []
    if not rows:
        return [], ["文件为空或没有数据行"]
    # 表头校验
    unknown = [h for h in (rows[0].keys() if rows else [])
    if h not in FIELD_ALIASES]
```

```
    if unknown:                                                    5915
        errors.append("存在无法识别的列，将被忽略：%s" %                    5916
"、".join(map(str, unknown)))                                      5917
    for idx, raw in enumerate(rows, start=2):       # 第 1 行是表头  5918
        rec = {}                                                   5919
        for k, v in raw.items():                                   5920
            key = FIELD_ALIASES.get(str(k).strip())                5921
            if key:                                                5922
                rec[key] = v                                       5923
        unit_type = str(rec.get("unit_type") or "").strip()        5924
        if not unit_type:                                          5925
            errors.append("第 %d 行：缺少「单元类型」" % idx)          5926
            continue                                               5927
        if unit_type not in unit_type_map:                         5928
            errors.append("第 %d 行：图斑内不存在单元类型「%s」（现有：%s）"  5929
                          % (idx, unit_type,                       5930
"、".join(unit_type_map) or "无"))                                 5931
            continue                                               5932
        if unit_type not in UNIT_TYPES:                            5933
            errors.append("第 %d 行：单元类型「%s」不是系统支持的三种之一"    5934
                          % (idx, unit_type))                      5935
            continue                                               5936
        fields, heavy = {}, {}                                     5937
        ok = True                                                  5938
        for key, val in rec.items():                               5939
            if key in ("unit_type",):                              5940
                continue                                           5941
            if key.startswith("hm_"):                              5942
                n, err = _num(val)                                 5943
                if err:                                            5944
                    errors.append("第 %d 行：%s %s" % (idx,        5945
key[3:], err))                                                     5946
                    ok = False                                     5947
                elif n is not None:                                5948
                    heavy[key[3:]] = n                             5949
                continue                                           5950
            if key == "aspect":                                    5951
                s = str(val).strip() if val is not None            5952
else ""                                                            5953
                if s == "":                                        5954
                    fields["aspect"] = None                        5955
                elif s in ASPECT_TYPES:                            5956
                    fields["aspect"] = s                           5957
                else:                                              5958
                    errors.append("第 %d 行：坡向「%s」无效（应为 %s）"  5959
                                  % (idx, s,                       5960
"、".join(ASPECT_TYPES)))                                          5961
                    ok = False                                     5962
                continue                                           5963
            n, err = _num(val)                                     5964
```

```
                if err:
                    errors.append("第 %d 行:%s %s" % (idx, key,
    err))
                    ok = False
                else:
                    fields[key] = n
            if ok:
                parsed.append({"unit_type": unit_type,
    "fields": fields,
                               "heavy_metals": heavy})
    return parsed, errors
# ==== app/services/indicator_service.py ====
# -*- coding: utf-8 -*-
"""指标定义维护（M6）。
把论文表 2/表 3 的指标体系写入 indicators 表，使指标名称、单位、
理想值与取值依据可在界面上查看维护，而不再只存在于代码常量中。
代码常量（app.utils.constants）仍是计算时的**权威取值来源**；
本表用于展示、核对与记录取值依据。二者由 seed_indicators() 保持同步。
"""
from app.models import db, Indicator
from app.utils.constants import (
    UNIT_TYPES, UNIT_ENABLED_INDICATORS, IDEAL_SOIL_RATIO,
    IDEAL_THICKNESS,
    IDEAL_PH_RANGE, IDEAL_ORGANIC, IDEAL_COVERAGE,
    IDEAL_SHANNON,
    HEAVY_METAL_LIMIT, SLOPE_BREAK, ASPECT_SCORES,
)
ALL_UNITS = ",".join(UNIT_TYPES)
# (code, name, level, parent, unit, ideal_value,
    ideal_type, enabled_units, desc, is_constraint)
TREE = [
    # ---- 一级指标 ----
    ("soil", "土壤指标", 1, None, None, None, None, "碎石边坡,平台",
    "论文表 2 评价指标之一级指标", False),
    ("veg", "植被指标", 1, None, None, None, None, ALL_UNITS,
    "论文表 2 评价指标之一级指标", False),
    ("terrain", "地形指标", 1, None, None, None, None,
    "岩质边坡,碎石边坡", "论文表 2 评价指标之一级指标", False),
    # ---- 二级指标：土壤 ----
    ("soil_ratio", "土石比", 2, "soil", "—",
    str(IDEAL_SOIL_RATIO), "benefit", "碎石边坡,平台",
     "论文 4.2.2 节：理想值 0.8，根据矿山修复实际情况确定", False),
    ("thickness", "土层厚度", 2, "soil", "cm",
    str(IDEAL_THICKNESS), "benefit", "碎石边坡,平台",
     "论文 4.2.2 节：理想值 20 cm", False),
    ("ph", "pH", 2, "soil", "—", "%g~%g" % IDEAL_PH_RANGE,
    "mid", "碎石边坡,平台",
     "论文 4.2.2 节：pH 用适中指标的标准化方法，理想区间 %g~%g" %
    IDEAL_PH_RANGE, False),
    ("organic_matter", "有机质", 2, "soil", "g/kg",
```

```
        str(IDEAL_ORGANIC), "benefit", "碎石边坡,平台",
          "《全国第二次土壤普查养分等级划分标准》二等（较丰富）最低值 30 g/kg", False),
        ("pollution", "污染指标", 2, "soil", "mg/kg", "见三级指标",
        "cost", "碎石边坡,平台",
          "同时作为限制性指标与评价指标；8 种重金属等权合成（各 0.125）", True),
        # ---- 二级指标：植被 ----
        ("coverage", "植被覆盖度", 2, "veg", "%",
        str(IDEAL_COVERAGE), "benefit", ALL_UNITS,
          "论文 4.2.2 节：参照生态系统取值 75%", False),
        ("shannon", "生物多样性", 2, "veg", "—", str(IDEAL_SHANNON),
        "benefit", ALL_UNITS,
          "Shannon-Wiener 多样性指数，参照生态系统取值 0.7", False),
        # ---- 二级指标：地形 ----
        ("slope", "坡度", 2, "terrain", "°", "%g° 分界" %
        SLOPE_BREAK, "cost", "岩质边坡,碎石边坡",
          "论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号：25°
        为陡坡重要分界", False),
        ("aspect", "坡向", 2, "terrain", "—",
        "阳坡0.2/阴阳坡0.6/阴坡1.0", "discrete", "岩质边坡,碎石边坡",
          "论文 4.2.2 节：阴坡及半阴坡水分条件更优，故赋值高于阳坡", False),
]
# 污染指标下的三级指标（8 种重金属）
for _m, _limit in HEAVY_METAL_LIMIT.items():
    TREE.append((
        "hm_" + _m, _m, 3, "pollution", "mg/kg",
    str(_limit), "cost",
        "碎石边坡,平台",
        "《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值 %g mg/kg" %
    _limit,
        False,
    ))
def seed_indicators():
    """写入/更新指标定义（幂等）。返回写入条数。"""
    code_map = {
        "benefit": "越大越优", "cost": "越小越优",
        "mid": "适中型", "discrete": "离散赋值",
    }
    n = 0
    for (code, name, level, parent, unit, ideal, itype,
    units, desc, is_con) in TREE:
        ind = Indicator.query.filter_by(code=code).first()
        if ind is None:
            ind = Indicator(code=code)
            db.session.add(ind)
        ind.name = name
        ind.level = level
        ind.parent_code = parent
        ind.unit = unit
        ind.ideal_value = ideal
        ind.ideal_type = itype
```

```
            ind.enabled_units = units                                    6065
            ind.description = desc                                       6066
            ind.is_constraint = is_con                                   6067
            ind.is_evaluative = (level in (2, 3))                        6068
            n += 1                                                       6069
        db.session.commit()                                              6070
        return n                                                         6071
def ideal_type_label(code):                                              6072
        return {"benefit": "越大越优", "cost": "越小越优",                  6073
                "mid": "适中型", "discrete": "离散赋值"}.get(code,          6074
        code or "—")                                                     6075
#  ==== app/templates/ahp/experts.html ====                             6076
{% extends "base.html" %}                                                6077
{% block title %}多专家判断矩阵{% endblock %}                               6078
{% block content %}                                                      6079
<div class="card">                                                       6080
  <h2>多专家判断矩阵 · 录入与汇总</h2>                                        6081
  <p class="muted">                                                      6082
    论文明确权重由「组织 １０ 位专家对各项指标权重进行打分」确定。                  6083
    此页支持多位专家分别录入判断矩阵，再用几何平均汇总，                          6084
    保持矩阵互反性，并重新做一致性检验。                                      6085
  </p>                                                                   6086
  <form method="get" class="actions" style="margin-bottom:14 6087
    px;">                                                                6088
    <div>                                                                6089
      <label class="field__label" for="unit-type">微地质单元类型</l 6090
    abel>                                                                6091
      <select id="unit-type" name="unit_type"                            6092
    onchange="this.form.submit()">                                       6093
        {% for t in ["岩质边坡", "碎石边坡", "平台"] %}                      6094
        <option value="{{ t }}" {% if unit_type == t                     6095
    %}selected{% endif %}>{{ t }}</option>                               6096
        {% endfor %}                                                     6097
      </select>                                                          6098
    </div>                                                               6099
    <div>                                                                6100
      <label class="field__label" for="level">指标层级</label>           6101
      <select id="level" name="level"                                    6102
    onchange="this.form.submit()">                                       6103
        <option value="1" {% if level == 1 %}selected{%                  6104
    endif %}>一级指标</option>                                           6105
        <option value="2" {% if level == 2 %}selected{%                  6106
    endif %}>二级指标</option>                                           6107
      </select>                                                          6108
    </div>                                                               6109
  </form>                                                                6110
  <p class="muted">                                                      6111
    当前范围：<strong>{{ unit_type }}</strong> ·                         6112
    {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif    6113
    %}。                                                                 6114
```

```
    若选二级指标，请先到                                                    6115
    <a href="/ahp/">权重计算</a> 选择一级指标后进入本页。                    6116
  </p>                                                                      6117
  {% if rows %}                                                             6118
  <table>                                                                   6119
    <caption class="muted">已录入 {{ rows|length }}                         6120
    位专家的矩阵</caption>                                                  6121
    <thead>                                                                 6122
      <tr>                                                                  6123
        <th scope="col">专家</th>                                           6124
        <th scope="col">矩阵规模</th>                                       6125
        <th scope="col">该矩阵 CR</th>                                      6126
        <th scope="col">一致性</th>                                         6127
        <th scope="col">录入时间</th>                                       6128
        <th scope="col">操作</th>                                           6129
      </tr>                                                                 6130
    </thead>                                                                6131
    <tbody>                                                                 6132
      {% for r in rows %}                                                   6133
      <tr>                                                                  6134
        <td>{{ r.expert_name }}</td>                                        6135
        <td class="num">{{ n }} × {{ n }}</td>                              6136
        <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is                  6137
    not none else "—" }}</td>                                               6138
        <td>                                                                6139
          {% if r.cr is not none and r.cr < 0.1 %}<span                     6140
    class="tag tag--ok">通过</span>                                         6141
          {% else %}<span class="tag tag--warn">不通过</span>{%             6142
    endif %}                                                                6143
        </td>                                                               6144
        <td class="mono nowrap">{{ r.created_at.strftime("%Y               6145
    -%m-%d %H:%M") if r.created_at else "—" }}</td>                         6146
        <td>                                                                6147
          <form method="post" action="/ahp/experts/{{ r.id                  6148
    }}/delete"                                                              6149
                onsubmit="return confirm('确定删除专家「{{                  6150
    r.expert_name }}」的矩阵吗？');">                                       6151
            {{ form.hidden_tag() }}                                         6152
            <button type="submit" class="btn btn--sm                        6153
    btn--danger">删除</button>                                              6154
          </form>                                                           6155
        </td>                                                               6156
      </tr>                                                                 6157
      {% endfor %}                                                          6158
    </tbody>                                                                6159
  </table>                                                                  6160
  <p class="actions" style="margin-bottom:10px;">                           6161
    <a class="btn" href="/ahp/experts/aggregate?unit_type={{                6162
    unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{%                 6163
    endif %}">                                                              6164
```

```
        汇总计算（几何平均）
    </a>
</p>
{% else %}
<p class="muted">尚未录入任何专家矩阵。请在下方录入第一位专家的判断矩阵。</p>
{% endif %}
<h3>录入一位专家的判断矩阵</h3>
<form method="post">
    {{ form.hidden_tag() }}
    <input type="hidden" name="action" value="add">
    <input type="hidden" name="unit_type" value="{{
    unit_type }}">
    <input type="hidden" name="level" value="{{ level }}">
    {% if l1 %}<input type="hidden" name="l1" value="{{ l1
    }}">{% endif %}
    <div class="field">
      <label class="field__label" for="expert-name">专家姓名</la
    bel>
      <input id="expert-name" type="text" name="expert_name"
             style="width:220px;" placeholder="如：张工">
    </div>
    <table style="width:auto;">
      <thead>
        <tr>
          <th></th>
          {% for name in names %}<th scope="col">{{ name
    }}</th>{% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for i in range(n) %}
        <tr>
          <th scope="row">{{ names[i] }}</th>
          {% for j in range(n) %}
            {% if i == j %}
            <td class="muted num">1</td>
            {% elif i < j %}
            <td>
              <label class="muted" style="position:absolute;
    left:-9999px;"
                     for="e-{{ i }}-{{ j }}">{{ names[i] }}
    相对 {{ names[j] }} 的重要性</label>
              <input id="e-{{ i }}-{{ j }}" type="number"
    step="0.01" min="0.01"
                     name="m_{{ i }}_{{ j }}" value="{{
    default[i][j] }}">
            </td>
            {% else %}
            <td class="muted num">{{ default[i][j] }}</td>
            {% endif %}
```

```
            {% endfor %}
          </tr>
          {% endfor %}
        </tbody>
      </table>
      <p class="actions" style="margin-top:12px;">
        <button type="submit" class="btn">录入该专家矩阵</button>
        <a class="btn btn--ghost" href="/ahp/indicators">指标维护</a>
        <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/ahp/experts_aggregate.html ====
{% extends "base.html" %}
{% block title %}多专家矩阵汇总{% endblock %}
{% block content %}
<div class="card">
  <h2>多专家矩阵汇总 · {{ unit_type }} · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{% endif %}</h2>
  <p class="muted">
    对各位专家的判断矩阵按对应元素取几何平均：
    a<sub>ij</sub> = (a<sub>ij</sub><sup>(1)</sup> × … × a<sub>ij</sub><sup>(k)</sup>)<sup>1/k</sup>。
    该方式可保持矩阵的互反性，汇总后重新计算权重与一致性比例。
  </p>
  {% if not result %}
  <p class="muted">
    该范围内尚未录入专家矩阵。
    <a href="/ahp/experts?unit_type={{ unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1 }}{% endif %}">去录入</a>
  </p>
  {% else %}
  <div class="stat-row">
    <div>
      <div class="stat__label">参与汇总的专家</div>
      <div class="stat__value">{{ result.count }} 位</div>
    </div>
    <div>
      <div class="stat__label">λmax</div>
      <div class="stat__value mono">{{ result.lambda_max }}</div>
    </div>
    <div>
      <div class="stat__label">CI</div>
      <div class="stat__value mono">{{ "%.4f"|format(result.ci) }}</div>
    </div>
```

```
    <div>                                                                    6265
      <div class="stat__label">CR（要求 &lt; 0.1）</div>                      6266
      <div class="stat__value mono">{{                                      6267
    "%.4f"|format(result.cr) }}</div>                                        6268
    </div>                                                                   6269
    <div>                                                                    6270
      <div class="stat__label">一致性检验</div>                              6271
      <div class="stat__value">                                              6272
        {% if result.consistent %}<span class="ok">通过</span>               6273
        {% else %}<span class="warn">不通过</span>{% endif %}                6274
      </div>                                                                 6275
    </div>                                                                   6276
  </div>                                                                     6277
  <h3>汇总后权重</h3>                                                        6278
  <table style="width:auto;">                                                6279
    <thead>                                                                  6280
      <tr>                                                                   6281
        <th scope="col">指标</th>                                            6282
        {% for name in names %}<th scope="col">{{ name                       6283
    }}</th>{% endfor %}                                                      6284
      </tr>                                                                  6285
    </thead>                                                                 6286
    <tbody>                                                                  6287
      <tr>                                                                   6288
        <th scope="row">权重</th>                                            6289
        {% for w in result.weights %}<td class="mono">{{ w                   6290
    }}</td>{% endfor %}                                                      6291
      </tr>                                                                  6292
    </tbody>                                                                 6293
  </table>                                                                   6294
  <h3>各专家矩阵的一致性</h3>                                                6295
  <table style="width:auto;">                                                6296
    <thead>                                                                  6297
      <tr><th scope="col">专家</th><th scope="col">该矩阵                    6298
    CR</th><th scope="col">判定</th></tr>                                    6299
    </thead>                                                                 6300
    <tbody>                                                                  6301
      {% for r in rows %}                                                    6302
      <tr>                                                                   6303
        <td>{{ r.expert_name }}</td>                                         6304
        <td class="mono">{{ "%.4f"|format(r.cr) if r.cr is                   6305
    not none else "—" }}</td>                                                6306
        <td>                                                                 6307
          {% if r.cr is not none and r.cr < 0.1 %}<span                      6308
    class="ok">通过</span>                                                   6309
          {% else %}<span class="warn">不通过</span>{% endif %}              6310
        </td>                                                                6311
      </tr>                                                                  6312
      {% endfor %}                                                           6313
    </tbody>                                                                 6314
```

```
  </table>
  {% if not result.consistent and result.most_inconsistent
    %}
  <p style="padding:12px;border-radius:6px;background:var(--
    danger-soft);
              border-left:4px solid var(--danger);">
    <strong class="warn">汇总矩阵一致性不通过，建议优先调整：</strong>
    第 {{ result.most_inconsistent.i + 1 }} 行、第 {{
    result.most_inconsistent.j + 1 }} 列的比较
    （当前 <span class="mono">{{ result.most_inconsistent.value
    |round(3) }}</span>，
    理想约 <span class="mono">{{ result.most_inconsistent.expec
    ted|round(3) }}</span>）。
    该偏差通常来自个别专家打分，可先核对一致性较差的专家矩阵。
  </p>
  {% endif %}
  {% endif %}
  <p class="actions" style="margin-top:16px;">
    <a class="btn btn--ghost" href="/ahp/experts?unit_type={
    { unit_type }}&level={{ level }}{% if l1 %}&l1={{ l1
    }}{% endif %}">返回录入</a>
    <a class="btn btn--ghost" href="/ahp/">返回权重计算</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/ahp/indicator_edit.html ====
{% extends "base.html" %}
{% block title %}编辑指标 · {{ ind.name }}{% endblock %}
{% block content %}
<div class="card" style="max-width:680px;">
  <h2>编辑指标 · {{ ind.name }}</h2>
  <p class="muted">
    指标编码 <span class="mono">{{ ind.code }}</span>、层级 {{
    ind.level }} 级，
    由论文指标体系确定，不可修改。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="name">指标名称</label>
      <input id="name" type="text" name="name"
    style="width:100%;" required
             value="{{ ind.name }}">
    </div>
    <div class="field">
      <label class="field__label" for="unit">计量单位</label>
      <input id="unit" type="text" name="unit"
    style="width:180px;"
             value="{{ ind.unit or '' }}">
    </div>
```

```
<div class="field">
  <label class="field__label" for="ideal-type">标准化类型</label>
  <select id="ideal-type" name="ideal_type">
    {% for code, label in [("benefit","越大越优"),("cost","越小越优"),("mid","适中型"),("discrete","离散赋值")] %}
    <option value="{{ code }}" {% if ind.ideal_type == code %}selected{% endif %}>{{ label }}</option>
    {% endfor %}
  </select>
</div>
<div class="field">
  <label class="field__label" for="ideal-value">理想取值</label>
  <input id="ideal-value" type="text" name="ideal_value" style="width:100%;"
         value="{{ ind.ideal_value or '' }}">
  <p class="muted" style="margin:4px 0 0;">
    如土石比 0.8、土层厚度 20、pH 5~9、植被覆盖度 75。
  </p>
</div>
<fieldset style="border:1px solid var(--line);border-radius:6px;padding:10px 14px;margin:0 0 12px;">
  <legend style="padding:0 6px;font-weight:600;">启用该指标的微地质单元</legend>
  {% for t in unit_types %}
  <label class="checkbox" style="margin-right:18px;">
    <input type="checkbox" name="enabled_units" value="{{ t }}"
           {% if t in ind.units_list() %}checked{% endif %}>
    <span>{{ t }}</span>
  </label>
  {% endfor %}
  <p class="muted" style="margin:8px 0 0;">
    论文设定：岩质边坡不启用土壤指标，平台不启用地形指标。
  </p>
</fieldset>
<div class="field">
  <label class="checkbox" for="is-constraint">
    <input id="is-constraint" type="checkbox" name="is_constraint" value="1"
           {% if ind.is_constraint %}checked{% endif %}>
    <span>同时作为限制性指标（参与一票否决判定）</span>
  </label>
</div>
<div class="field">
  <label class="field__label" for="description">取值依据说明</label
```

```
        <textarea id="description" name="description"
      style="min-height:88px;">{{ ind.description or ''
      }}</textarea>
      </div>
      <p class="actions" style="margin-top:16px;">
        <button type="submit" class="btn">保存</button>
        <a class="btn btn--ghost" href="/ahp/indicators">返回指标列
      表</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/ahp/indicators.html ====
{% extends "base.html" %}
{% block title %}指标定义维护{% endblock %}
{% block content %}
<div class="card">
  <h2>指标体系维护</h2>
  <p class="muted">
    按论文表 2 与表 3 建立的指标树。此处可核对指标名称、单位、标准化类型、
    理想取值与取值依据，并按需调整启用该指标的微地质单元。
    计算时的权威取值仍以 <span class="mono">app/utils/constants.py</sp
    an> 为准，
    本页用于展示与核对。
  </p>
  <p class="actions" style="margin-bottom:14px;">
    <a class="btn btn--sm btn--ghost"
    href="/ahp/">返回权重计算</a>
    <a class="btn btn--sm btn--ghost"
    href="/ahp/experts">多专家矩阵</a>
    <form method="post" action="/ahp/indicators/reset"
    style="display:inline;"
          onsubmit="return confirm('将按论文取值重置全部指标定义，自定义修改会丢失。
    确定继续？');">
      {{ form.hidden_tag() }}
      <button type="submit" class="btn btn--sm
    btn--danger">按论文取值重置</button>
    </form>
  </p>
  {% for level, title in [(1, "一级指标"), (2, "二级指标"), (3,
    "三级指标")] %}
  {% set level_items = all_ind|selectattr("level",
    "equalto", level)|list %}
  {% if level_items %}
  <h3>{{ title }}（{{ level_items|length }} 项）</h3>
  <table>
    <thead>
      <tr>
        <th scope="col">指标</th>
        <th scope="col">上级指标</th>
```

```
            <th scope="col">单位</th>
            <th scope="col">标准化类型</th>
            <th scope="col">理想取值</th>
            <th scope="col">启用单元</th>
            <th scope="col">限制性</th>
            <th scope="col">操作</th>
          </tr>
        </thead>
        <tbody>
          {% for i in level_items %}
          <tr>
            <td style="text-align:left;">
              <span class="mono muted">{{ i.code
        }}</span><br>{{ i.name }}
            </td>
            <td>{{ (all_ind|selectattr("code", "equalto",
        i.parent_code)|list|first).name if i.parent_code else
        "—" }}</td>
            <td>{{ i.unit or "—" }}</td>
            <td>{{ type_label(i.ideal_type) }}</td>
            <td class="mono">{{ i.ideal_value or "—" }}</td>
            <td style="text-align:left;">{{ i.enabled_units or
        "—" }}</td>
            <td>
              {% if i.is_constraint %}<span class="tag
        tag--warn">是</span>
              {% else %}<span class="tag tag--muted">否</span>{%
        endif %}
            </td>
            <td><a href="/ahp/indicators/{{ i.code
        }}/edit">编辑</a></td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
      {% endif %}
      {% endfor %}
    </div>
    {% endblock %}
    # ==== app/templates/auth/edit_user.html ====
    {% extends "base.html" %}
    {% block title %}编辑用户 · {{ user.username }}{% endblock %}
    {% block content %}
    <div class="card" style="max-width:560px;">
      <h2>编辑用户 · {{ user.username }}</h2>
      <form method="post">
        {{ form.hidden_tag() }}
        <div class="field">
          <label class="field__label" for="role">角色</label>
          <select id="role" name="role">
```

```
          {% for code, label in role_labels.items() %}
          <option value="{{ code }}" {% if user.role == code
      %}selected{% endif %}>{{ label }}</option>
          {% endfor %}
        </select>
      </div>
      <div class="field">
        <label class="checkbox" for="active">
          <input id="active" type="checkbox" name="active"
      value="1"
                 {% if user.active %}checked{% endif %}>
          <span>启用该账号</span>
        </label>
        <p class="muted" style="margin:6px 0 0;">
          停用后该账号无法登录，但其历史操作记录会保留。
          系统至少需保留一名启用状态的管理员。
        </p>
      </div>
      <div class="field">
        <label class="field__label" for="password">重置密码（留空则不修改
      ）</label>
        <input id="password" type="password" name="password"
               autocomplete="new-password"
      style="width:280px;"
               placeholder="留空表示保持原密码">
      </div>
      <p class="actions" style="margin-top:18px;">
        <button type="submit" class="btn">保存</button>
        <a class="btn btn--ghost" href="/users">返回</a>
      </p>
    </form>
</div>
{% endblock %}
# ==== app/templates/auth/logs.html ====
{% extends "base.html" %}
{% block title %}操作日志{% endblock %}
{% block content %}
<div class="card">
  <h2>操作日志</h2>
  <p class="muted">
    系统自动记录所有写操作（新建、编辑、删除、导入、执行评价等），
    便于追溯数据变更。失败的操作也会记录并标注 HTTP 状态码。
  </p>
  <p class="actions" style="margin-bottom:12px;">
    <a class="btn btn--sm btn--ghost"
    href="/users">返回用户管理</a>
  </p>
  {% if not records %}
  <p class="muted">暂无操作记录。</p>
  {% else %}
```

```
  <table>
    <caption class="muted">共 {{ total }} 条记录，当前第 {{ page }} / {{ pages }} 页</caption>
    <thead>
      <tr>
        <th scope="col">时间</th>
        <th scope="col">操作人</th>
        <th scope="col">操作</th>
        <th scope="col">对象</th>
        <th scope="col">说明</th>
      </tr>
    </thead>
    <tbody>
      {% for r in records %}
      <tr>
        <td class="mono nowrap">{{ r.created_at.strftime("%Y-%m-%d %H:%M:%S") if r.created_at else "—" }}</td>
        <td>{{ users_map.get(r.user_id, "—") }}</td>
        <td style="text-align:left;">{{ r.action }}</td>
        <td>
          {% if r.target_type %}{{ r.target_type }}{% if r.target_id %}#{{ r.target_id }}{% endif %}
          {% else %}—{% endif %}
        </td>
        <td style="text-align:left;">{{ r.detail or "—" }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
  {% if pages > 1 %}
  <nav class="actions" aria-label="日志分页">
    {% if page > 1 %}
    <a class="btn btn--sm btn--ghost" href="/logs?page={{ page - 1 }}">上一页</a>
    {% endif %}
    <span class="muted">第 {{ page }} / {{ pages }} 页</span>
    {% if page < pages %}
    <a class="btn btn--sm btn--ghost" href="/logs?page={{ page + 1 }}">下一页</a>
    {% endif %}
  </nav>
  {% endif %}
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/error.html ====
{% extends "base.html" %}
{% block title %}{{ code }} · {{ message }}{% endblock %}
{% block content %}
```

```
<div class="card" style="max-width:560px;">
  <h2>{{ code }} · {{ message }}</h2>
  <p class="muted">
    请返回上一页重试；若确认该操作应当可用，请检查当前账号的角色权限。
  </p>
  <p class="actions" style="margin-top:16px;">
    <a class="btn" href="/">返回首页</a>
  </p>
</div>
{% endblock %}
# ==== app/templates/indicator/import.html ====
{% extends "base.html" %}
{% block title %}批量导入指标数据{% endblock %}
{% block content %}
<div class="card" style="max-width:860px;">
  <h2>批量导入指标数据</h2>
  <p class="muted">
    支持 CSV 与 Excel（.xlsx/.xls）。文件每行对应一个微地质单元，
    系统按「单元类型」匹配图斑内已划分的单元后写入数据。
    先下载模板按格式填写，可减少出错。
  </p>
  <p class="actions" style="margin-bottom:14px;">
    <a class="btn btn--sm btn--ghost"
    href="/indicator/import/template">下载导入模板（CSV）</a>
    <a class="btn btn--sm btn--ghost"
    href="/indicator/">返回指标数据</a>
  </p>
  <form method="post" enctype="multipart/form-data">
    {{ form.hidden_tag() }}
    <div class="field">
      <label class="field__label" for="patch-id">目标矿山图斑</lab
    el>
      <select id="patch-id" name="patch_id" required>
        <option value="">— 请选择 —</option>
        {% for p in patches %}
        <option value="{{ p.id }}">{{ p.name }} ({{
    p.units.count() }} 个单元)</option>
        {% endfor %}
      </select>
    </div>
    <div class="field">
      <label class="field__label" for="datafile">数据文件</label
    >
      <input id="datafile" type="file" name="datafile"
             accept=".csv,.xlsx,.xls" required>
    </div>
    <p class="actions" style="margin-top:16px;">
      <button type="submit" class="btn">开始导入</button>
    </p>
  </form>
```

```
  <h3>模板列说明</h3>
  <p class="muted">
    共 {{ headers|length }} 列。留空表示该指标「不适用」。
    坡向须填「阳坡 / 阴阳坡 / 阴坡」之一。未检测的重金属留空即可
    （按论文注释视为未受污染）。
  </p>
  <p class="mono muted" style="line-height:1.9;">{{
    headers|join(" , ") }}</p>
  <h3>导入规则</h3>
  <ul class="muted" style="padding-left:20px;line-height:1.9
    ;">
    <li>「单元类型」必须与图斑内已划分的单元一致，否则该行会被跳过并在结果中提示。</li>
    <li>同一单元类型若在文件中出现多行，以最后一行为准。</li>
    <li>已有数据的单元会被覆盖更新；没有数据的单元将新建记录。</li>
    <li>数值列请填纯数字，不要带单位（如填 <span class="mono">10</span> 而非
    <span class="mono">10cm</span>）。</li>
  </ul>
</div>
{% if report %}
<div class="card" style="max-width:860px;">
  <h2>导入结果</h2>
  <div class="stat-row">
    <div>
      <div class="stat__label">目标图斑</div>
      <div style="font-size:15px;">{{ report.patch }}</div>
    </div>
    <div>
      <div class="stat__label">成功导入</div>
      <div class="stat__value ok">{{ report.saved }}
    个单元</div>
    </div>
    <div>
      <div class="stat__label">问题行</div>
      <div class="stat__value {% if report.errors %}warn{%
    endif %}">{{ report.errors|length }}</div>
    </div>
  </div>
  {% if report.errors %}
  <h3>未导入的行</h3>
  <ul class="muted" style="padding-left:20px;line-height:1.9
    ;">
    {% for e in report.errors %}<li>{{ e }}</li>{% endfor %}
  </ul>
  {% else %}
  <p class="ok" style="margin-top:12px;">全部数据行均已成功导入。</p>
  {% endif %}
  <p class="actions" style="margin-top:16px;">
    <a class="btn btn--ghost" href="/indicator/">查看指标数据</a>
  </p>
</div>
```

```
{% endif %}
{% endblock %}
# ==== app/templates/report/print.html ====
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<title>自然恢复适宜性评价报告 · {{ patch.name }}</title>
<style>
  @page { size: A4; margin: 18mm 16mm; }
  * { box-sizing: border-box; }
  body {
    margin: 0; color: #23262b; background: #fff;
    font-family: "Microsoft YaHei", "SimSun", sans-serif;
    font-size: 10.5pt; line-height: 1.65;
  }
  h1 { font-size: 17pt; text-align: center; margin: 0 0
    6px; }
  h2 { font-size: 13pt; color: #1f3a5f; margin: 20px 0 8px;
       border-bottom: 1px solid #dfe3e8; padding-bottom:
    4px; }
  h3 { font-size: 11.5pt; color: #1f3a5f; margin: 16px 0
    6px; }
  .sub { text-align: center; color: #6b7280; font-size:
    9.5pt; margin-bottom: 18px; }
  table { width: 100%; border-collapse: collapse; margin:
    8px 0 14px;
          font-variant-numeric: tabular-nums; }
  th, td { border: 1px solid #c9ccd1; padding: 4px 8px;
    text-align: center; }
  th { background: #eef2f7; font-weight: 600; }
  .left { text-align: left; }
  .mono { font-family: Consolas, "Courier New", monospace; }
  .kv { width: auto; }
  .kv th { width: 30%; text-align: left; }
  .kv td { text-align: left; }
  ul { margin: 6px 0 12px; padding-left: 20px; }
  .note { color: #6b7280; font-size: 9.5pt; }
  .chart { margin: 10px 0 16px; }
  .sig { margin-top: 28px; font-size: 10pt; }
  .avoid-break { break-inside: avoid; }
  svg { max-width: 100%; }
</style>
</head>
<body>
  <h1>历史遗留废弃矿山自然恢复适宜性评价报告</h1>
  <p class="sub">
    {{ patch.name }}
    {% if patch.admin_region %} · {{ patch.admin_region
    }}{% endif %}
```

```
    <br>历史遗留废弃矿山自然恢复适宜性评价系统 V1.0 ·
    生成时间 {{ generated_at }}
</p>
<h2>一、图斑基本信息</h2>
<table class="kv">
    <tr><th>图斑名称</th><td>{{ patch.name }}</td></tr>
    <tr><th>所在行政区</th><td>{{ patch.admin_region or "—"
    }}</td></tr>
    <tr><th>图斑面积（公顷）</th><td>{{ patch.area_ha if
    patch.area_ha is not none else "—" }}</td></tr>
    <tr><th>开采矿种</th><td>{{ patch.mineral_type or "—"
    }}</td></tr>
    <tr><th>开采方式</th><td>{{ patch.mining_method or "—"
    }}</td></tr>
    <tr><th>责任主体缺失</th><td>{{ "是" if patch.responsible_entit
    y_missing else "否" }}</td></tr>
    {% if reference %}
    <tr><th>参照生态系统</th>
        <td>植被覆盖度基准 {{ reference.coverage_benchmark }}%,
            多样性指数基准 {{ reference.shannon_benchmark
    }}</td></tr>
    {% endif %}
</table>
<h2>二、评价结论</h2>
<table class="kv">
    <tr><th>最终评价因子 F</th><td class="mono"><strong>{{
    "%.4f"|format(result.final_score) }}</strong></td></tr>
    <tr><th>适宜性等级</th><td><strong>{{ result.grade
    }}</strong></td></tr>
    <tr><th>判定依据</th><td>{{ result.grade_info.meaning
    }}</td></tr>
    <tr><th>推荐措施</th><td>{{ result.grade_info.measure
    }}</td></tr>
</table>
<h2>三、各微地质单元得分</h2>
<table>
    <thead>
      <tr><th>微地质单元</th><th>面积占比</th><th>单元评价因子
    F'</th><th>加权贡献</th></tr>
    </thead>
    <tbody>
      {% for u in result.unit_results %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td class="mono">{{ "%.2f%%"|format(u.ratio * 100)
    }}</td>
        <td class="mono">{{ "%.4f"|format(u.unit_score)
    }}</td>
        <td class="mono">{{ "%.4f"|format(u.weighted)
    }}</td>
```

```
        </tr>
        {% endfor %}
    </tbody>
</table>
<div class="chart">{{ chart_units|safe }}</div>
<h2>四、一级指标权重分布</h2>
<div class="chart">{{ chart_weight|safe }}</div>
<h2>五、得分明细</h2>
{% for u in result.unit_results %}
<h3>{{ u.unit_type }}（占比 {{ "%.2f%%"|format(u.ratio *
    100) }}，单元得分 {{ "%.4f"|format(u.unit_score) }}）</h3>
<table>
    <thead>
        <tr>
            <th>一级指标</th><th>二级指标</th><th>实测值</th>
            <th>标准化值</th><th>权重</th><th>贡献</th>
        </tr>
    </thead>
    <tbody>
        {% for d in u.details %}
        <tr>
            <td>{{ d.l1 }}</td>
            <td class="left">{{ d.l2 }}</td>
            <td class="mono left">{{ "—" if d.raw is none else
    d.raw }}</td>
            <td class="mono">{{ "—" if d.std_value is none else
    "%.4f"|format(d.std_value) }}</td>
            <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
            <td class="mono">{{ "—" if d.contribution is none
    else "%.4f"|format(d.contribution) }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>
<p class="note">
    一级指标得分：
    {% for k, v in u.l1_scores.items() %}{{ k }} = {{
    "%.4f"|format(v) }}{% if not loop.last %}；{% endif %}{%
    endfor %}
</p>
{% endfor %}
<h2>六、评价依据说明</h2>
<ul class="note">
    <li>指标标准化按论文式（８） 执行，按指标性质分为越大越优、越小越优、
        适中型与离散赋值四类。</li>
    <li>单元评价因子按论文式（９） 由二级指标权重加权求得；
        整矿评价因子由各微地质单元按面积占比加权合成。</li>
    <li>适宜性等级按论文表 4 划分：F&lt;０．３ 不适宜、０．３≤F≤０．７ 基本适宜、F≥０．７
    适宜。</li>
    <li>指标权重取自论文表 3，按岩质边坡、碎石土边坡、平台三类微地质单元分别确定。</li>
```

```
    </ul>
    <p class="sig">
      评价人（签字）：_____  日期：_____
    </p>
</body>
</html>
# ==== app/utils/__init__.py ====
# -*- coding: utf-8 -*-
# ==== app/utils/browser.py ====
# -*- coding: utf-8 -*-
"""HTML 转 PDF（M9 报告导出）。
复用本机已安装的 Chrome/Edge 无头模式打印，避免引入 reportlab、
weasyprint 等额外依赖。生成过程使用系统临时目录中的 ASCII 文件名，
避免中文路径在 file:// URL 下出问题。
"""
import os
import shutil
import subprocess
import tempfile
import uuid
CANDIDATES = [
    r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe",
    r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
    r"C:\Program Files\Microsoft\Edge\Application\msedge.exe",
]
# 允许通过环境变量覆盖，便于在其他机器上指定浏览器
ENV_KEY = "DSH_BROWSER_PATH"
def browser_path():
    """定位可用的浏览器可执行文件，找不到返回 None。"""
    override = os.environ.get(ENV_KEY)
    if override and os.path.isfile(override):
        return override
    for p in CANDIDATES:
        if os.path.isfile(p):
            return p
    for name in ("chrome", "msedge", "chromium"):
        found = shutil.which(name)
        if found:
            return found
    return None
def html_to_pdf(html, timeout=90):
    """把 HTML 字符串渲染为 PDF，返回字节；失败返回 (None, 原因)。"""
    exe = browser_path()
    if exe is None:
        return None, ("未找到可用的浏览器（Chrome/Edge），无法生成 PDF。"
```

```
                                "可设置环境变量 %s 指向浏览器可执行文件。" % ENV_KEY)
    tmpdir = tempfile.mkdtemp(prefix="dshpdf_")
    tag = uuid.uuid4().hex[:8]
    html_path = os.path.join(tmpdir, "r_%s.html" % tag)
    pdf_path = os.path.join(tmpdir, "r_%s.pdf" % tag)
    try:
        with open(html_path, "w", encoding="utf-8") as f:
            f.write(html)
        cmd = [
            exe, "--headless=new", "--disable-gpu",
    "--no-sandbox",
            "--no-first-run", "--no-default-browser-check",
            "--no-pdf-header-footer",
            "--virtual-time-budget=4000",
            "--print-to-pdf=%s" % pdf_path,
            "file:///" + html_path.replace("\\", "/"),
        ]
        p = subprocess.run(cmd, capture_output=True,
    text=True,
                           timeout=timeout,
    encoding="utf-8", errors="replace")
        if not os.path.isfile(pdf_path) or
    os.path.getsize(pdf_path) < 800:
            return None, "浏览器未能生成 PDF（%s）" % (p.stderr or
    "").strip()[:200]
        with open(pdf_path, "rb") as f:
            return f.read(), None
    except subprocess.TimeoutExpired:
        return None, "生成 PDF 超时，请稍后重试"
    except Exception as e:
        return None, "生成 PDF 失败：%s" % e
    finally:
        shutil.rmtree(tmpdir, ignore_errors=True)
# ==== app/utils/charts.py ====
# -*- coding: utf-8 -*-
"""轻量图表（M9）。
用纯 SVG 输出，不依赖任何绘图库或前端脚本：
- 服务端渲染，页面与打印（PDF）中都能稳定显示
- 每个图元都带数值标签，黑白打印时仍可读（不单靠颜色区分）
- 单色方案，符合界面「一个视图一个强调色」的约定
"""
from html import escape
ACCENT = "#1f3a5f"
ACCENT_SOFT = "#7f93ad"
INK = "#23262b"
MUTED = "#6b7280"
LINE = "#dfe3e8"
# 灰度梯度：黑白打印时相邻扇区 / 条柱仍可区分
GRAYS = ["#1f3a5f", "#456188", "#6d86a8", "#96a9c4",
    "#bcc9da", "#dbe3ec"]
```

```
def _fmt(v, digits=4):                                                  6965
    try:                                                                6966
        return ("%%.%df" % digits) % float(v)                           6967
    except (TypeError, ValueError):                                     6968
        return str(v)                                                   6969
def hbars(items, width=620, bar_h=22, gap=12, label_w=150,              6970
    value_w=76):                                                        6971
    """水平条形图。items: [(标签, 数值, 显示文本)]"""                    6972
    if not items:                                                       6973
        return '<p class="muted">暂无数据</p>'                          6974
    n = len(items)                                                      6975
    height = n * (bar_h + gap) + 10                                     6976
    max_v = max((abs(float(v)) for _, v, _ in items),                   6977
    default=1) or 1                                                     6978
    body_w = width - label_w - value_w                                  6979
    parts = ['<svg viewBox="0 0 %d %d" width="100%%"                    6980
    height="%d" '                                                       6981
             'role="img" aria-label="条形图" '                          6982
             'style="max-width:%dpx;font-family:inherit;">'             6983
    % (width, height, height, width)]                                   6984
    for i, (label, value, disp) in enumerate(items):                    6985
        y = i * (bar_h + gap) + 5                                       6986
        frac = (abs(float(value)) / max_v) if max_v else 0              6987
        w = max(2.0, body_w * frac)                                     6988
        fill = GRAYS[i % len(GRAYS)]                                    6989
        parts.append(                                                   6990
            '<text x="%d" y="%d" text-anchor="end"                      6991
    font-size="12" fill="%s">%s</text>'                                 6992
            % (label_w - 10, y + bar_h * 0.72, INK,                     6993
    escape(str(label))))                                                6994
        parts.append(                                                   6995
            '<rect x="%d" y="%d" width="%.1f" height="%d"               6996
    rx="3" fill="%s"/>'                                                 6997
            % (label_w, y, w, bar_h, fill))                             6998
        parts.append(                                                   6999
            '<text x="%d" y="%d" font-size="12" fill="%s" '             7000
            'style="font-variant-numeric:tabular-nums;">%s</            7001
    text>'                                                              7002
            % (label_w + w + 8, y + bar_h * 0.72, MUTED,                7003
    escape(str(disp))))                                                 7004
    parts.append('</svg>')                                              7005
    return "".join(parts)                                               7006
def donut(items, size=210, thickness=34):                               7007
    """环形图（权重分布）。items: [(标签, 数值, 显示文本)]              7008
    图例中给出数值与占比，黑白打印时靠图例与灰度深浅区分。              7009
    """                                                                 7010
    vals = [(l, float(v), d) for l, v, d in items if                    7011
    float(v) > 0]                                                       7012
    if not vals:                                                        7013
        return '<p class="muted">暂无数据</p>'                          7014
```

```
    total = sum(v for _, v, _ in vals) or 1                                7015
    r = (size - thickness) / 2                                             7016
    cx = cy = size / 2                                                     7017
    import math                                                            7018
    parts = ['<svg viewBox="0 0 %d %d" width="%d"                          7019
height="%d" role="img" '                                                   7020
             'aria-label="权重分布环形图">' % (size, size, size,           7021
size)]                                                                     7022
    angle = -math.pi / 2                                                   7023
    for i, (label, v, disp) in enumerate(vals):                            7024
        sweep = 2 * math.pi * (v / total)                                  7025
        a0, a1 = angle, angle + sweep                                      7026
        x0, y0 = cx + r * math.cos(a0), cy + r *                           7027
math.sin(a0)                                                               7028
        x1, y1 = cx + r * math.cos(a1), cy + r *                           7029
math.sin(a1)                                                               7030
        large = 1 if sweep > math.pi else 0                                7031
        parts.append(                                                      7032
            '<path d="M %.2f %.2f A %.2f %.2f 0 %d 1 %.2f                  7033
%.2f" '                                                                    7034
            'fill="none" stroke="%s" stroke-width="%d"/>'                  7035
            % (x0, y0, r, r, large, x1, y1, GRAYS[i %                      7036
len(GRAYS)], thickness))                                                   7037
        angle = a1                                                         7038
    parts.append('<text x="%d" y="%d" text-anchor="middle"                 7039
font-size="12" '                                                           7040
                 'fill="%s">合计</text>' % (cx, cy - 2,                    7041
MUTED))                                                                    7042
    parts.append('<text x="%d" y="%d" text-anchor="middle"                 7043
font-size="15" '                                                           7044
                 'fill="%s" style="font-variant-numeric:tabu               7045
lar-nums;">%s</text>'                                                      7046
                 % (cx, cy + 18, INK, _fmt(total, 4)))                     7047
    parts.append('</svg>')                                                 7048
    legend = ['<table style="width:auto;margin-top:10px;"><t               7049
head><tr>'                                                                 7050
              '<th scope="col">项目</th><th                                7051
scope="col">数值</th>'                                                     7052
              '<th scope="col">占比</th></tr></thead><tbody>']             7053
    for i, (label, v, disp) in enumerate(vals):                            7054
        legend.append(                                                     7055
            '<tr><td style="text-align:left;">'                            7056
            '<span style="display:inline-block;width:10px;he               7057
ight:10px;'                                                                7058
            'border-radius:2px;background:%s;margin-right:6p               7059
x;"></span>%s</td>'                                                        7060
            '<td class="mono">%s</td><td                                   7061
class="mono">%.1f%%</td></tr>'                                             7062
            % (GRAYS[i % len(GRAYS)], escape(str(label)),                  7063
escape(str(disp)),                                                         7064
```

```
                    100.0 * v / total))
        legend.append('</tbody></table>')
        return ('<div style="display:flex;flex-wrap:wrap;gap:20px;align-items:center;">'
                + "".join(parts) + "".join(legend) + '</div>')
# ==== app/utils/schema.py ====
# -*- coding: utf-8 -*-
"""轻量结构迁移。
SQLite 下 `db.create_all()` 只建缺失的表，不会给已有表补列。
本模块在启动时检查目标列是否存在，缺失则 ALTER TABLE 补上，
使旧数据库无需重建即可用上新字段。
"""
from sqlalchemy import text
# （表名， 列名， 列定义）
REQUIRED_COLUMNS = [
    # M2 遥感影像挂接
    ("mine_patches", "image_path", "VARCHAR(256)"),
    ("mine_patches", "image_note", "VARCHAR(256)"),
    ("mine_patches", "image_uploaded_at", "DATETIME"),
    # M6 指标维护
    ("indicators", "enabled_units", "VARCHAR(128)"),
    ("indicators", "description", "TEXT"),
    # M6 多专家矩阵
    ("expert_matrices", "unit_type", "VARCHAR(32)"),
    ("expert_matrices", "l1_name", "VARCHAR(64)"),
    ("expert_matrices", "cr", "FLOAT"),
]
def ensure_schema(db):
    """为已有表补齐缺失列。返回补列清单。"""
    added = []
    try:
        with db.engine.connect() as conn:
            for table, column, ddl in REQUIRED_COLUMNS:
                rows = conn.execute(text("PRAGMA table_info(%s)" % table)).fetchall()
                if not rows:
                    continue                                # 表还不存在，create_all 会建
                existing = {r[1] for r in rows}
                if column in existing:
                    continue
                conn.execute(text(
                    "ALTER TABLE %s ADD COLUMN %s %s" % (table, column, ddl)))
                conn.commit()
                added.append("%s.%s" % (table, column))
    except Exception as e:                                  # 迁移失败不应阻断启动
        print("[schema] 结构检查跳过：%s" % e)
    return added
```

```
# ==== app/utils/uploads.py ====
# -*- coding: utf-8 -*-
"""上传文件处理（M2 遥感影像、M7 批量导入）。"""
import os
import re
import uuid
from datetime import datetime
from werkzeug.utils import secure_filename
def _safe_ext(filename):
    return os.path.splitext(filename)[1].lower().lstrip(".")
def ext_allowed(filename, allowed):
    return _safe_ext(filename) in allowed
def save_upload(file_storage, subdir, allowed_ext,
    upload_root):
    """保存上传文件，返回相对 static 的路径（如 uploads/patches/xxx.png）。
    文件名重命名为「时间戳_随机串.扩展名」，避免中文名与重名问题。
    """
    if file_storage is None or not file_storage.filename:
        return None, "未选择文件"
    ext = _safe_ext(file_storage.filename)
    if ext not in allowed_ext:
        return None, "不支持的文件类型：.%s" % ext
    target_dir = os.path.join(upload_root, subdir)
    os.makedirs(target_dir, exist_ok=True)
    # 仅保留安全字符，并加时间戳与随机串防重名
    base = secure_filename(os.path.splitext(file_storage.fil
    ename)[0]) or "file"
    base = re.sub(r"[^A-Za-z0-9_\-]", "", base)[:40] or
    "file"
    name = "%s_%s.%s" % (datetime.now().strftime("%Y%m%d%H%M
    %S"),
                         uuid.uuid4().hex[:6], ext)
    abs_path = os.path.join(target_dir, name)
    file_storage.save(abs_path)
    rel = os.path.join("uploads", subdir,
    name).replace("\\", "/")
    return rel, None
def remove_upload(rel_path, upload_root):
    """删除已保存的上传文件（相对 static 的路径）。"""
    if not rel_path:
        return
    abs_path = os.path.join(upload_root,
    rel_path.replace("uploads/", "", 1))
    try:
        if os.path.isfile(abs_path):
            os.remove(abs_path)
    except OSError:
        pass
# ==== app/views/__init__.py ====
# -*- coding: utf-8 -*-
```